Course: LING285: Human Language and Technology
Title of Lecture: Lecture 11
Date: Tuesday, October 6
Professor: Khalil Iskarous

Section 1: One Mass, Two Directions

- A **mass-spring system** is a mass hooked up to a spring, and its behavior comes from what each part "wants." **The spring wants to be at its equilibrium position**, which is its resting length, so if it is stretched or squished it pulls or pushes to get back there. **A mass that is moving wants to keep on moving**, so once it gets going it does not stop by itself.
  - When the mass is pulled out and let go, the spring pulls it back toward equilibrium. The mass is moving when it gets there, so it **keeps on moving** past equilibrium and squishes the spring.
  - The squished spring now pushes the mass back the other way, the mass overshoots again, and the cycle repeats. That repeating back and forth is **vibration**.
  - On a real table, **friction** slowly brings the motion to a stop. The model leaves friction out, so in the model the system vibrates forever.
- The same system can move in **two directions**, and it is important to understand both kinds of motion, because some systems are easier to think about one way than the other.
  - In **longitudinal** motion, the mass moves **left and right**, back and forth along the line of the spring.
  - In **transverse** motion, the mass sits between **two walls** with a spring on each side, and it moves **up and down**, across the line of the springs.
- **Both kinds of motion work in exactly the same way, and for the same reason.** Pulling the transverse mass up **stretches the springs**, and since springs hate being stretched, they pull the mass back down.
  - As the mass passes the middle, the springs are not stretched at all. The mass is still moving, though, so it keeps going and stretches them on the other side.
  - Both versions vibrate at the **same rate**, because they have the same mass and the same total stiffness in the direction of motion. Changing the direction of motion does not change the rate.

Section 2: Mass and Stiffness Change the Rate

- **Frequency** is the number of back-and-forth cycles a system completes each second, and two things set it, the **mass** and the **stiffness**. Each change has the same effect in longitudinal motion and in transverse motion.
- **A heavier mass vibrates more slowly.** A heavy mass is hard to get moving and hard to turn around, so each trip takes longer. **Heavy branches sway back and forth slowly, while little branches sway quickly.**
  - Making the mass **four times heavier cuts the rate in half**.
- **A stiffer spring makes the system vibrate faster.** The stiffer something is, the harder it pulls back toward where it started, so it turns the mass around sooner.
  - Making the spring **four times stiffer doubles the rate**.
- The two rules point in opposite directions. **Raising the mass lowers the frequency, and raising the stiffness raises the frequency.**

- Each part has its effect only while it is doing its job. **A mass affects frequency by moving, and a spring affects frequency by stretching and compressing.** A mass that never moves could be made a thousand pounds heavier without changing the frequency at all, and a spring that never stretches adds no stiffness.

Section 3: Two Masses, Two Modes

- With **two masses**, the system has two walls, a spring from each wall to a mass, and **one spring between the two masses**. All the masses are equal, and all the springs have the same stiffness.
- This system can vibrate in **two different ways**, which are called **modes**. **A mode is just another word for a way of vibrating**, and the modes are told apart by asking whether the masses go **together** or **against each other**.
  - In **Mode 1**, the masses move **together**. In transverse motion, both are pulled up and let go, and they rise and fall side by side. In longitudinal motion, both are pulled to the left, and they move left and right as a pair.
  - In **Mode 2**, the masses move **against each other**. One is pulled down and the other up, so whenever one rises the other falls. In longitudinal motion, one is pulled left and the other right, so they move toward each other and then apart.
- A system can also vibrate in **both modes at the same time**. Pulling both masses up, but one higher than the other, starts this mixed motion.
  - Both masses start to fall, which is Mode 1. Because they start at different heights, the middle spring also pushes them against each other, which is Mode 2.
  - The result goes **together and against each other at once**, so there are moments when both masses head the same way and moments when they head opposite ways. This seems strange, but real movement and real speech production behave this way all the time.

Section 4: Why the Second Mode Vibrates Faster

- The two modes have **different frequencies**. **When the masses go together, they move back and forth slowly, and when they go against each other, they move back and forth quickly**, so **Mode 2 has the higher frequency**.
- This is puzzling at first, because both modes use **the same two masses and the same three springs**. The explanation comes from asking **what each spring is actually doing** in each mode.
  - In both modes, the **two outer springs** stretch and compress as the masses move.
  - In **Mode 1**, the masses move together, so the **middle spring never stretches or compresses**. It just rides along like a rigid bar, so **effectively Mode 1 has only two springs**.
  - In **Mode 2**, the masses move against each other, so the middle spring is stretched and squished over and over. **All three springs are working**, so Mode 2 has three units of stiffness instead of two.
- **More stiffness means more pull back toward equilibrium, and more pull means a higher frequency.** Nothing about the springs themselves changed. What changed is **how each mode uses the springs**, and that difference alone makes Mode 2 faster.
- The general rule follows directly. **The more the masses go against each other, the more the springs stretch and compress, so the more effective stiffness a mode**

has and the higher its frequency. This is why a system with several masses has several frequencies. It has several modes, and each higher mode has higher stiffness.

Section 5: Three Masses, Three Modes

- With **three masses** and four springs, the system has **three modes**, one more than with two masses.
  - In **Mode 1**, all three masses go back and forth **together**, and this mode has the **lowest frequency**.
  - In **Mode 2**, the **middle mass stays still** while the two outer masses move against each other, and this mode has the **middle frequency**. To start it, the outer masses are pulled apart very carefully without touching the middle one.
  - In **Mode 3**, the **middle mass moves opposite to the two outer masses**, and this mode has the **highest frequency**. It is easiest to see in transverse motion, where the middle mass drops while both outer masses rise.
- The frequencies follow the same stiffness reasoning.
  - **Mode 1 is very slow** because the masses move together, so the **two middle springs do nothing**. Their job is to stretch and compress, and when they just ride along they add no stiffness.
  - In **Mode 2**, the springs do stretch and compress, but less than they could.
  - In **Mode 3**, **everything goes against everything else**, so it has the most stretching and compressing, the most stiffness, and the fastest motion.
- Because a mass acts only by moving, **changing one mass affects only the modes in which that mass moves**. Making the **middle mass heavier lowers the frequency of Mode 1 and Mode 3**, since the middle mass moves in both of them. **Mode 2 stays the same**, because in Mode 2 the middle mass does not move at all.
- All three modes can also happen **at the same time**, and however the system is started, it vibrates as some mix of these **pure modes**.
  - **Learning a musical instrument means learning how to make the masses and springs vibrate in certain modes and not others.** How a string is plucked or a violin is bowed decides which modes get excited, and the favored modes are heard as **notes**.
  - A **trained singer** does the same with the vocal folds, controlling their mass and stiffness well enough to carry a tune.

Section 6: One More Mass, One More Mode

- The pattern holds for any number of masses. **N masses give N independent modes**, so one mass has one way to vibrate, two masses have two, and six masses have six.
  - Moving to higher modes, more parts move against each other, so the pattern has **more reversals**, which are places where neighboring masses head in opposite directions.
  - More reversals mean more stretching, so **higher modes have higher frequencies**.
  - In chains of separate masses like these, the mode frequencies are generally **not whole-number multiples** of the lowest frequency.
- The same mode looks alike no matter how many masses the chain has.
  - **Every Mode 1 is alike**, because all the masses go up and down together.

    - **Every Mode 2 is alike**, because the **left side goes against the right side**. With two masses, that simply means one mass against the other, and with an odd number of masses, the middle one stays still.
    - **Every Mode 3 is alike**, because the **middle goes against the sides**.
    - With **four masses**, the four modes are all four together, the left pair against the right pair, the middle pair against the outer pair, and every mass against its neighbors.
- Packing in more and more masses and springs, all the way to an **infinite number**, gives a **string**, like a guitar or piano string. **Every little piece of a string has mass, and every little piece stretches like a spring**, so a string has an infinite number of modes.
    - In the **lowest mode**, the whole string goes up and down together with hardly any stretching between its pieces, so it gives the **lowest note**.
    - In each higher mode, more and more parts of the string move against each other.
    - A string plucked without any skill vibrates in many modes at once.

Section 7: A String's Modes and Its Spectrum

- A **string fixed at both ends** with its lowest mode at **100 hertz** has its next modes at **200, 300, 400 and 500 hertz**, and **hertz** means times per second. Each higher mode completes more cycles in the same amount of time.
    - These frequencies are **whole-number multiples** of the lowest one, and modes that line up this way are called **harmonics**. Here they are **consecutive harmonics**, which include both the odd and the even multiples.
- The even spacing comes from how each mode is built out of the first one. **Mode 1 is a single bump**, and **Mode 2 is two copies of that bump**, with one half going up while the other half goes down.
    - With two copies, Mode 2 has **twice the stiffness** of Mode 1, so it has twice the frequency. Mode 3 fits three copies and has three times the stiffness, and the fifth mode has **the first mode fitted into it five times**.
    - So each mode adds one more unit of stiffness, and the frequencies climb in equal steps. With a lowest mode of 200 hertz, the fifth mode is five times 200.
- A real string vibrates in **all of its modes at the same time**. Adding the separate motions together at every point gives one complicated, changing shape, and that combined motion is what an actual plucked string does.
- The **spectrum** of a sound tells **which frequencies are in it and how much of each there is**. **Frequency** runs along the horizontal axis, and **amplitude**, or energy, runs up the vertical axis.
    - For the string moving in all five modes, the spectrum has **five evenly spaced spikes**, one at each mode's frequency. In this example the spikes have **equal height**, since nothing has filtered the sound to strengthen some frequencies and weaken others.
    - If the string vibrated **only in Mode 3**, the other modes would have no motion, so the spectrum would have **one spike at the frequency of Mode 3** and nothing else. That pattern is like a **one-hot encoding**, with one active slot and all the others at zero.

Section 8: The Vocal Folds as a Membrane

- **The vocal folds are even more complicated than strings.** They are a sheet of muscle, which is called a **membrane**, so their masses and springs spread across a surface instead of along a line. The same logic of modes still applies.

    - In **Mode 1**, the whole surface goes up and down together.
    - In **Mode 2**, the **left half goes against the right half**.
    - In **Mode 3**, the surface splits the other way, and the two halves on either side of a still middle line go against each other.
    - In **Mode 4**, four sections go against each other, and still higher modes get more and more complicated. Each mode is labeled by a pair of numbers that counts how many bumps fit along the length and how many fit across the width.
- In real vocal folds, **all the modes happen at the same time**, so voicing makes the folds vibrate in many modes at once and produces a spectrum full of spikes.
    - The evenly spaced frequencies attached to these membrane modes are teaching values.
    - The vocal folds are held fixed along their edges, so they **behave like a string**, which has evenly spaced harmonics.

Section 9: Theory, Prediction and Experiment

- **The scientific method** depends on **very clear statements of how the world works**, usually written as **differential equations**, which describe how quantities change over time. **Solving those equations gives a prediction** of what the world will be like when it is actually measured.
    - The spring equations are an example. Solving them predicts that a spring that is pulled goes back, and a spring that is pushed also goes back.
- For the voice, the reasoning moves from a theory to a prediction.
    - The **theory** is that the **vocal folds are a membrane**, so when they vibrate, the vibration has **equidistant peaks in its spectrum**. These evenly spaced peaks are called **harmonics**.
    - The equations for the vocal folds are the same type as the spring equations, only many more of them and slightly more complicated, so the intuition from masses and springs is enough to reason with.
    - The **prediction** is that **anyone's laryngeal vibration signal, when measured, will have a spectrum with equidistant harmonics.**
- The prediction was tested by recording two voices in **Praat**, a program for analyzing speech. Each speaker made a sound with the **mouth wide open**, so that no particular vowel was being made.
    - In the first voice, the lowest real spike was near **100 hertz**, and the next spikes fell close to two, three, four and five times that value, **evenly spaced just as predicted**.
    - In the second voice, the lowest spike was near **220 hertz**, and the next ones fell near twice and three times that value, again evenly spaced but farther apart.
    - **The prediction held for both voices**, so a theory built from simple masses and springs made a precise and correct prediction about real speech.
- The difference between the two voices follows from the mass rule. The second voice has a **higher pitch**, so those vocal folds are probably **smaller and thinner**. **Thinner folds have less mass, and less mass means a higher frequency**, so the lowest harmonic is higher and every harmonic after it is spaced farther apart.
- This is part of how people are **recognized by their voices**. Each person's vocal folds give their voice its own spectrum, and becoming familiar with someone includes becoming familiar with that spectrum.

- Twins share the same genetics and similar vocal folds, so they can be hard to tell apart on the phone.
- How a person says particular vowels also matters, but that depends on the vibration of the **air** above the vocal folds, not on the folds themselves.

Section 10: One Fixed End and One Free End

- A system can also be **fixed at one end and free at the other**. A **building** is like this, because it is fixed to the ground at the bottom and free to sway at the top in high winds or an earthquake. A string can be set up the same way, **fixed at the bottom** and **free at the top**.
  - The **fixed bottom** cannot move, so every mode has a point of **no motion** there, which is called a **node**.
  - The **free top** moves the most, so every mode has its point of **largest motion** there, which is called an **antinode**.
- This end condition changes which modes are possible. **Only the odd harmonics occur**, so the modes come at **100, 300, 500 and 700 hertz**, which are one, three, five and seven times the lowest frequency.
  - **Mode 1** has no still point inside the string. Its motion grows steadily from the fixed bottom to the free top, like a building swaying as a whole.
  - The odd steps come from fitting copies of Mode 1 together. Two copies would put a still point at both ends, which would make the system **fixed at both ends**, so that pattern is impossible here.
  - **Three copies** do end with free motion at the top, so the next mode holds three copies and has three times the stiffness and three times the frequency of Mode 1. The mode after that holds five copies.
  - When all four modes are excited equally, the spectrum measured at the free end has **four equal spikes**.
- **Civil engineers design buildings to sway only in Mode 1**, with every floor moving together. A building is not made so stiff that it cannot move at all, since a building that cannot give at all fares worse in an earthquake. In higher modes, some floors would move one way while others moved the other way, which would be dangerous, so the design keeps the swaying in the lowest mode.
- For a string of the **same length and the same wave speed**, switching from fixed at both ends to fixed at one end and free at the other **halves the lowest frequency**.
  - When **both ends are fixed**, the string has **every whole-number multiple** of the lowest frequency.
  - When **one end is fixed and the other is free**, it has **only the odd multiples**.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 12
Date: Tuesday, October 13
Professor: Khalil Iskarous

Section 1: Dynamic Systems in the Economy

- The **2026 Nobel Prize in economics** went to **Robert Hall** and **Michael Woodford** for the **microeconomic foundations of macroeconomic stabilization**. **Stabilization** means getting a system to its equilibrium and keeping it there, which is the same job that **balancing feedback** does and the same thing a spring does when it pulls a mass back to rest.
  - In their model of the economy, **unemployment and inflation rise and fall in turns**, like the two curves of an oscillator. One curve plays the role of the position, which starts high, goes negative and comes back, and the other plays the role of the velocity, which is the flip of it.
  - The aim of stabilization policy is to **even out these peaks and troughs**, so that the swings get smaller instead of producing high unemployment or runaway inflation.
- Woodford's theory is called **Dynamic Stochastic General Equilibrium**, or **DSGE**, and each word in the name has a meaning that carries over from dynamical systems.
  - **Dynamic** means the economy is a system that changes over time according to rules.
  - **Stochastic** means the model can handle **random shocks**, such as a war starting, a factory burning down or a large company's CEO being caught embezzling. This is close to stochastic gradient descent, where one odd data point can push the estimate far off, but later data pull it back toward the average.
  - **Equilibrium** means the goal is a steady state that people can live in, without wild jumps.
- **Macroeconomic** quantities describe a whole economy, like inflation or the number of people unemployed, while **microeconomic** events are single choices, like one person buying something. Explaining large patterns from many tiny events is the same kind of problem as speech, where many tiny vibrations in the mouth add up to a meaning a listener understands without tracking any of them.

Section 2: Masses, Springs and Their Modes

- A mass-spring system vibrates because **the spring wants to return to equilibrium** while **a moving mass wants to keep moving**. The same vibration can happen **longitudinally**, left and right along the spring, or **transversely**, up and down between two walls, and both kinds of motion behave in exactly the same way. Understanding both is useful, because the vocal folds and the air in the vocal tract each vibrate most clearly in a different direction.
  - **Raising the mass lowers the frequency**, so the system goes back and forth fewer times per second.
  - **Raising the stiffness raises the frequency**, so the system goes back and forth more times per second.
- With **two masses** there are **two modes**. In **Mode 1** the masses move together, and in **Mode 2** they move against each other. The system can also do both at once, with the masses rising and falling together while also moving against each other.
  - **Mode 1 is slower** because the middle spring just rides along without stretching or compressing, so effectively it has **only two springs**.

    - **Mode 2 is faster** because all **three springs** stretch and compress, and more working springs means **more stiffness** and a **higher frequency**.
    - Making the **middle spring stiffer** while the system vibrates leaves **Mode 1 unchanged**, since that spring never stretches in Mode 1. The same change makes **Mode 2 rise**, since in Mode 2 the middle spring stretches and compresses all the time.
- With **three masses** there are **three modes**. In Mode 1 all three move together, in Mode 2 the middle mass is frozen while the outer two move against each other, and in Mode 3 the middle mass moves against both outer masses. The more the springs stretch and compress, the higher the mode's frequency.
    - Making **Mass 2**, the middle mass, heavier **lowers the frequency of Mode 1 and Mode 3**, because the middle mass moves in both.
    - The same change has **no effect on Mode 2**, because in Mode 2 the middle mass does not move at all.
    - **A mass affects frequency only through its movement.** If it does not move, it has no effect, if it moves a little, it has a small effect, and if it moves a lot, it has a large effect.

Section 3: More Masses and the String

- **Every added mass adds one more mode**, and the same patterns repeat as the chain grows.
    - **Mode 1** is always everything going up and down together.
    - **Mode 2** always splits the system into **two parts** that go against each other. With an odd number of masses, such as three, five or seven, the middle mass is frozen in Mode 2.
    - **Mode 3** always splits the system into **three parts**, with the middle going one way and the sides going the other.
    - Higher modes have more parts going against each other, so the springs stretch and compress far more, and those modes vibrate faster.
- Packing in more and more masses and springs gives a **string**, like the string of a guitar or the strings inside a piano. **Every little piece of a string has weight, which is its mass, and every little piece hates being stretched or compressed, which is its springiness.** The same mode patterns appear in the string, with everything moving together in Mode 1, the two halves going against each other in Mode 2, and the middle going against the sides in Mode 3.
    - In musical instruments, these frequencies are heard as **notes**.
    - The ear can pick out the notes of many instruments at the same time, because it is itself built of masses and springs that respond to the masses and springs vibrating in the world.

Section 4: Harmonics of a String Fixed at Both Ends

- A string fixed at both ends with its first mode at **100 hertz** has its next modes at **200, 300, 400 and 500 hertz**, so every mode is an **exact multiple** of the first. These multiples are called **harmonics**.
- The multiples come from **how many copies of the first mode fit into each higher mode**. The first mode is a single bump that swings from a maximum to a minimum and back.
    - The second mode holds **two** of those bumps, one side going up while the other goes down, so it has twice the frequency.
    - The third mode holds **three** bumps, and the fifth holds **five**, so the frequency keeps multiplying by the number of copies that fit.

    - So if the first mode is 100 hertz, the fifth mode is 500 hertz.
- A real string vibrates in **many modes at once**, and their sum gives an erratic-looking motion, much like the waveform of speech. A single mode by itself is a **pure tone**, which sounds so annoying that it is used for sirens. What we normally hear has many modes at the same time.
- The **spectrum** shows this mixture. **Frequency** runs along the horizontal axis, and **amplitude**, meaning how much vibration there is, runs up the vertical axis. For this string, the spectrum has spikes at 100, 200, 300, 400 and 500 hertz, which says that all five frequencies are present in the sound.
- The same reasoning predicts what happens when one piece of the string is made **heavier**, for example by thickening the very middle of the string.
    - In **Mode 1**, the middle moves up and down a lot, so the added mass **lowers** its frequency.
    - In **Mode 2**, the middle does not move at all, so the added mass changes **nothing**.
    - In **Mode 3** the middle moves again, so its frequency drops, and in **Mode 4** the middle is still again, so nothing changes. Going up the modes, the effect **alternates** between lowering the frequency and having no effect.
- **Motion and stretching alternate along a vibrating string.** Where a piece of the string moves the most, it is just carried up and down and is hardly stretched. Where the string is stretched the most, such as right next to a fixed end, there is **no movement** at all. Along the string, these places keep alternating, with the most movement and the least stretching at one spot, then the least movement and the most stretching at the next.
    - **The shortcut is that maximal springiness is at the points of no movement.** In **Mode 5** of a string fixed at both ends, the string does not move at the **two fixed ends** or at the **four points where it crosses its middle line**, so those **six places** have the most springiness.
    - The opposite question has the opposite answer. The **most motion**, and so the **least springiness**, is at the tops of the **five bumps**.

Section 5: Principles of Mass-Spring Systems

- All of these results follow from a short set of principles, and they are not separate facts to memorize, because each one follows from the others.
    - **Raising the mass lowers the frequency, and raising the stiffness raises the frequency.**
    - **A system with n masses has n modes of vibration**, and each higher mode has a higher frequency.
    - **An increase in mass lowers frequency in proportion to how much that mass moves**, and how much it moves depends on where it is.
    - **Where there is maximal movement there is no stretching, and where there is maximal stretching there is no movement.**
    - **An increase in stiffness raises frequency in proportion to how much that spring stretches and compresses**, which also depends on where it is.
    - In a **fixed-fixed system**, the harmonics are **multiples of the base frequency**, because the base mode fits a whole number of times into each successive mode.
    - In a **fixed-free system**, the frequencies are **odd multiples of the base frequency**, because the base mode fits an odd number of times into each successive mode.

Section 6: A Local Change Affects Two Modes Differently

- Changing one small piece of a string can push two modes in **opposite directions**. Consider a spot where **Mode 4 is moving the most**, and where **Mode 5 crosses its middle line**, so that Mode 5 is not moving there at all. Both the **mass** and the **stiffness** of that small piece are raised.
  - **For Mode 4**, the piece is moving, so the extra mass **lowers** the frequency. The piece is barely stretched, so the extra stiffness does nothing.
  - **For Mode 5**, the piece does not move, so the extra mass does nothing. The piece is stretched the most there, so the extra stiffness **raises** the frequency.
  - So the same change **lowers Mode 4's frequency and raises Mode 5's frequency**. Each mode has to be reasoned out on its own by asking whether the spot is moving or stretching in that mode.
- This way of reasoning about small local changes, called **perturbation theory**, was first worked out with **drums**, which are also made of countless masses and springs. In the late nineteenth century, before cameras could capture such fast motion, small amounts of **wax** were added to different spots on a drum.
  - Wax on a spot predicted to vibrate a lot added moving mass, and the drum's frequency was heard to **drop**.
  - Wax on a spot predicted not to move at all made **no difference** to the sound.

Section 7: The Vocal Folds as a Membrane

- The **larynx** holds the **vocal folds**, which form a **membrane** that is a bit like a drum, so they have many modes of vibration. In Mode 1 all the masses go up and down together, in Mode 2 the two halves go against each other, and the higher modes divide the surface into more and more parts.
- The **scientific method** depends on clear theories, usually written as **differential equations**, whose solutions give predictions that can be measured. The theory here is that the vocal folds vibrate in many modes at once, so a measured voice should have a spectrum with **equidistant harmonics**.
  - A voice recorded and examined with **Praat** had its lowest harmonic at roughly 130 hertz, and the next harmonics fell close to twice and three times that value. **The measurement matched the prediction.**
  - Each person's lowest frequency is different, but the harmonics above it are always close to multiples of it.
- Changes to the mass and stiffness of the vocal folds change the voice. People who smoke heavily can develop **nodules** on their vocal folds, which add mass at one spot and change the stiffness of nearby parts. That changes how the folds vibrate, which is part of why a heavy smoker's voice sounds different.

Section 8: One Fixed End and One Free End

- A **building** is a system that is **fixed on one side and free on the other**, because it is fixed to the earth at the bottom while the top can sway in very high winds or an earthquake.
  - Structural engineers design buildings to absorb that motion in **Mode 1**, with the whole building swaying together. On a high floor this can be unpleasant, but it is safe.
  - They design buildings to never vibrate in a higher mode, where some floors would go one way and other floors the other way, because that would break the structure.

- The **air inside the vocal tract** is also a fixed-free system. At the bottom, the **vocal folds are closed**, so the air cannot rush past them and **cannot move**, which makes that end **fixed**. At the lips, the air can go in and out easily, so that end is **free**. Unlike a building, the air in the vocal tract vibrates in **all of its modes**, and speaking changes the masses and springs of different parts of the air, which is how listeners know which vowel, consonant and word is being said.
- A fixed-free system has a different spectrum from a fixed-fixed one. **The frequencies jump by odd steps, at 100, 300, 500 and 700 hertz**, instead of climbing through every multiple.
  - The first mode goes from **no motion** at the fixed end to **lots of motion** at the free end, which is only a quarter of a full wave.
  - The next possible mode holds **three** of these no-motion-to-lots-of-motion pieces, so its frequency is three times the first. The mode after that holds **five**, so its frequency is five times the first.
  - Two pieces would end with no motion at both ends, which is a fixed-fixed pattern, so even multiples never occur.

Section 9: Air in a Fixed-Free Tube

- The air in the vocal tract can be modeled as a **chain of masses and springs** that is fixed at the glottis end and free at the lip end. The masses stand for the air's inertia and the springs stand for its springiness.
  - The lowest mode of this chain is like the **first formant**, **F1**, and the next mode is like the **second formant**, **F2**. The frequencies of a fixed-free system are called **formants**, while the frequencies of a fixed-fixed system are called harmonics.
  - The second mode oscillates **three times faster** than the first, because it holds three of the first mode's quarter-wave pieces.
  - The masses near the free end move very freely in both modes.
- Three places along the tube matter for vowels. A spot **one third of the way from the glottis** is where **/a/** is constricted, a spot **two thirds of the way from the glottis** is where **/i/** is constricted, and the **free end at the lips** is where **/u/** is constricted.

Section 10: Vowel Shapes and Constrictions

- Images of the vocal tract during five vowels show where the air is squeezed. A **constriction** is a place where the tongue or lips narrow the tract and squeeze the air. The **glottis** is the tiny space between the vocal folds at the bottom of the tract, so distances are measured from that fixed end.
  - In **done**, the central vowel, there is **no constriction**, and the airflow is basically unimpeded along the whole tract.
  - In **dean**, a **high front** vowel, the air is squeezed under the roof of the mouth, about **two thirds of the way from the glottis**.
  - In **Don**, a **low back** vowel, the air is squeezed far back in the throat, about **one third of the way from the glottis**.
  - In **dune**, a **high back** vowel made with **lip rounding**, the air is squeezed **at the lips**.
  - **Dan** is a **low front** vowel.

Section 11: What a Constriction Does to the Air

- **Constricting air raises both its stiffness and its mass.** Squeezing air is like squeezing the air in a tire, which makes the tire **stiffer**. Squeezing air also packs it into a smaller volume and raises its **density**, which makes the air more **massive** and harder to move.
- It might seem that raising mass and stiffness together would **cancel out**, since more mass lowers frequency and more stiffness raises it. **That is false** for any system with many masses and springs, and it is only true for a single mass on a single spring.
  - In a system with many masses and springs, each spot is either mostly **moving** or mostly **stretching** in a given mode.
  - At a spot with no motion and lots of stretching, the extra mass has no effect, so only the extra stiffness counts and the frequency **rises**.
  - At a spot with lots of motion and no stretching, the extra stiffness has no effect, so only the extra mass counts and the frequency **falls**.
  - Speech itself proves the effects do not cancel, because constrictions in different places clearly change the frequencies of the voice.

Section 12: /u/ Loads the Moving Lip End

- For **/u/**, rounding the lips squeezes the air at the **free end**, which raises the mass and the stiffness of the air there.
  - In **Mode 1**, the free end has lots of motion and almost no compression, so **motion dominates** and the extra mass lowers **F1**.
  - In **Mode 2**, the free end again has lots of motion, so **motion dominates** and the extra mass lowers **F2**.
  - The extra stiffness does not matter at either mode, because the air at the lips is moving, not stretching. So **both F1 and F2 go down**.
- This was tested by having a speaker start from the central vowel, whose formants were near 500, 1500 and 2500 hertz, and then move into **/u/**. **Both formants dropped**, as predicted, because the rounded lips added mass to the air right where it moves the most.

Section 13: /i/ Has Opposite Effects on F1 and F2

- For **/i/**, the air is squeezed about **two thirds of the way from the glottis**, and that one location acts differently in the two modes.
  - In **Mode 1**, this spot has lots of motion and hardly any compression, so **mass motion dominates**. The extra mass **lowers F1**.
  - In **Mode 2**, this spot has lots of compression, so **springiness dominates**. The extra stiffness **raises F2**.
  - So the same constriction pushes the two formants apart, with **F1 going down and F2 going up**.
- This was tested by having a speaker go from the central vowel into **/i/**. In the recording, the **first formant dropped a lot** and the **second formant went up**, just as the masses and springs predicted.

Section 14: /a/ Reverses the Two Effects

- For **/a/**, the air is squeezed about **one third of the way from the glottis**, and the pattern reverses.

    - In **Mode 1**, this spot near the fixed end has lots of compression, so **springiness dominates**. The extra stiffness **raises F1**.
    - In **Mode 2**, this spot has lots of motion, so **mass motion dominates**. The extra mass **lowers F2**.
    - So for **/a/**, **F1 goes up and F2 goes down**, the opposite of /i/, and the two formants come closer together.
- The general rule works in both directions. **If a constriction raises a formant a lot, the air at that spot must be springy and hardly moving in that mode**, so the added stiffness is what counted. **If a constriction lowers a formant a lot, the air at that spot must be moving a lot**, so the added mass is what counted.

Section 15: The Principles Applied to Air

- The same principles of mass-spring systems explain the vowels. **Mass up means frequency down and stiffness up means frequency up**, but each change matters only where it can act. **Extra mass matters where the air moves, and extra stiffness matters where the air compresses and expands.** Because the vocal tract is fixed at the glottis and free at the lips, its formants are **odd multiples** of the lowest one, since the quarter-wave base mode fits an odd number of times into each higher mode.

Section 16: Compression and Motion Alternate in an Air Column

- The air column can be modeled as **lines of air particles**, where each particle stands for billions of real air molecules. The closed vocal folds sit at the **fixed end**, and the open lips sit at the **free end**.
    - Where the lines of particles bunch close together and spread far apart, the air is **compressing and expanding**, which is a sign of **springiness**.
    - Where the lines just move back and forth without changing their spacing, the air is acting as a **mass**. This is easiest to see at the **free end**, where the air simply moves in and out.
    - At the fixed end the air cannot move, but it compresses and expands a lot.
- The three lowest modes of the air match the modes of a building. **F1** has a single point of no motion, at the fixed end. **F2** has two, one at the fixed end and one inside the tube. **F3** has three, and between every pair of no-motion points the air alternates between places of most motion and places of most compression.
- For the **schwa**, the central vowel with no constriction, the tube behaves like a uniform fixed-free system, so F2 should be three times F1 and F3 five times F1. A measured schwa had formants near **500, 1500 and 2500 hertz**, which are the odd multiples the theory predicts.

Section 17: Where Were You a Year Ago?

- A short question like **Where were you a year ago?** puts all of the constriction reasoning to work at once. Each sound squeezes the air at a different place, so the formants move up and down continuously instead of sitting at fixed values.
    - When the tongue makes an **/i/**-like squeeze near the front of the mouth, **F1 goes down and F2 goes up**, because that spot is massy in Mode 1 and springy in Mode 2.
    - When the lips round for **you**, **F1 and F2 both go down**, because the free end is massy in every mode.
    - When the tongue moves back and down, as in an **/a/**-like sound, **F1 rises and F2 falls**, because that spot is springy in Mode 1 and massy in Mode 2.

- **Speech is a continuous series of constrictions, not a list of separate vowels**, so the frequencies of the air in the vocal tract are always changing. The same principles of masses and springs predict which way each formant moves at every moment.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 13
Date: Thursday, October 15
Professor: Khalil Iskarous

Section 1: Principles of Mass-Spring Systems

- Every vibrating system in speech follows a small set of principles, and each one connects to the others.
  - **Raising the mass lowers the frequency, and raising the stiffness raises the frequency.**
  - **A system with n masses has n modes**, and each higher mode has a higher frequency, because more parts go against each other and more springs stretch and compress.
  - **An increase in mass lowers frequency in proportion to how much that mass moves.** A mass that does not move has zero effect, a mass that moves a little has a small effect, and a mass that moves a lot has a large effect.
  - **An increase in stiffness raises frequency in proportion to how much that spring stretches and compresses.** Both of these effects depend on where the change is made.
  - **Where there is maximal movement there is no stretching, and where there is maximal stretching there is no movement.**
  - In a **fixed-fixed system**, the frequencies are called **harmonics**, and they are **whole-number multiples** of the base frequency. In a **fixed-free system**, the frequencies are called **formants**, and they are **odd multiples** of the base frequency.
- These principles warn against a tempting mistake. If the mass and the stiffness of a system are both raised, it might seem the two effects simply **cancel out**. That is only true for a **single mass on a single spring**. In a system with many masses and springs, a spot that is moving a lot is hardly stretching, and a spot that is stretching a lot is hardly moving, so only one of the two changes matters at that spot.
  - Picture a spot on a string that never moves but is stretched hard. Extra mass there does nothing, while extra stiffness there raises the frequency, so the frequency goes up.
  - Picture a spot that swings up and down the most. Extra stiffness there does nothing, while extra mass slows it down, so the frequency goes down.

Section 2: More Masses, More Modes

- Adding masses adds modes, and the same shapes keep repeating. **Mode 1** always has everything moving together, **Mode 2** always splits the system into two halves going against each other, and **Mode 3** always has the middle going against the sides. The more the parts go against each other, the more the springs work, so **higher modes are stiffer and faster**.
  - In plain terms, a higher mode is not made of stiffer springs. It just makes the same springs do more stretching.

Section 3: Why Fixed-Fixed Frequencies Are Whole-Number Multiples

- In a string fixed at both ends, the first mode is **one half-wave**, a single bump that goes from no motion at one end, to the most motion in the middle, to no motion at the other end. Each higher mode is built by packing **more of these half-wave packets** into the same length.

    - **Mode 2 holds two packets**, with one half going up while the other half goes down. Each packet behaves like a copy of Mode 1, so Mode 2 has **twice the stiffness** and **twice the frequency**.
    - **Mode 3 holds three packets**, and **Mode 5 holds five**, so the fifth mode has five times the frequency of the first.
    - So the frequencies climb in equal steps. If the first harmonic is 100 hertz, the next ones are 200, 300, 400 and 500 hertz.
- The numbers do not have to start at 100. A voice whose base frequency is **250 hertz** has its next harmonics at 500, 750 and 1000 hertz, because each one is still a whole number of copies of the first.
    - The short version for an exam answer is that **the fifth mode has the first mode fitted into it five times**, so it has five times the stiffness and five times the frequency.

Section 4: Harmonics of Strings and Membranes

- A string fixed at both ends vibrates in all of these modes at once, and its **spectrum**, with **frequency** along the bottom and **amplitude** going up, has **evenly spaced spikes**, one for each harmonic. The **vocal folds** are fixed on all sides, just behind the Adam's apple, so they behave like this kind of fixed-fixed system and produce harmonics.
- The vocal folds form a **membrane**, and the same packet counting works on a surface. The first mode is a single bump across the whole membrane. The second mode squeezes two of those bumps in, so it has twice the frequency, and the fourth mode squeezes in four ups and downs, so it has four times the frequency.

Section 5: One Fixed End and One Free End

- A **building** is **fixed at the bottom** and **free at the top**. In high winds or an earthquake, engineers want it to sway only in **Mode 1**, all together. That is mildly unpleasant on a high floor but safe. A higher mode would send some floors one way and other floors the other way, and the whole structure could shatter.
- A string can be set up the same way, fixed at the bottom and free at the top. Its first mode is not a full half-wave anymore. It goes from **no motion at the fixed end** to **lots of motion at the free end**, which is only **one quarter of a wave**.

Section 6: Why Fixed-Free Frequencies Are Odd Multiples

- In a fixed-free system, each mode is built from **quarter-wave packets**, and each packet runs from no motion to lots of motion. **Mode 1 holds one packet**, **Mode 2 holds three**, **Mode 3 holds five** and **Mode 4 holds seven**.
    - So with a base of 100 hertz, the modes come at **1, 3, 5 and 7 times 100 hertz**, which are 100, 300, 500 and 700 hertz.
- **Two packets can never fit.** Stacking two quarter-wave packets gives no motion, then lots of motion, then no motion again. That puts a still point at the top as well as the bottom, which makes the system **fixed at both ends**, not fixed-free. Adding one more packet, for **three** in total, ends with lots of motion at the top again, so that is the next mode that is possible.
    - The simple version is that the free end must always be moving, and only an odd number of quarter-waves lands on a moving end.
- The frequencies of a fixed-free system are called **formants**. **A formant is a frequency of vibration of a system that is fixed on one side and completely free on the other**,

and formants are **odd multiples** of the lowest one. **F1** is the first formant, **F2** the second and **F3** the third.

Section 7: Counting Quarter-Waves

- Counting quarter-wave packets gives every formant from the first. If **F1 is 500 hertz**, then **F2 is three times that, 1500 hertz**, and **F3 is five times that, 2500 hertz**.
  - The steps between formants are twice the first formant, because each new mode adds two more quarter-wave packets.

Section 8: The Air in the Vocal Tract Is a Fixed-Free System

- The air inside the mouth and throat forms a tube. At one end are the **vocal folds**, and because they are closed, the air there **cannot go back and forth**, so that end is **fixed**. At the other end the **lips are open**, so the air can go back and forth freely, and that end is **free**.
  - So the air in the vocal tract is a fixed-free system, just like a building. **The motion of air in the vocal tract is like a building**, and the math describing them is almost exactly the same.
  - Its frequencies are therefore **formants**, which are odd multiples of the lowest one.
  - Speaking changes the stiffness and mass of the air in different parts of the tract, which moves the formants. That is how listeners hear different vowels and consonants.

Section 9: Compression and Motion Alternate in an Air Column

- The air in the tube can be modeled as lines of particles, where each particle stands for billions of air molecules.
  - Where the lines come **really close together and then really far apart**, the air is compressing and stretching, which means it is acting like a **spring**.
  - Where the lines just **move back and forth** without changing their distance from each other, the air is acting like a **mass**. The easiest place to see this is the **free end at the lips**, where the air simply moves in and out.
  - An easy way to read it is that bunching and spreading means springiness, and sliding without bunching means mass.
- The air's modes match a building's modes.
  - In **Mode 1**, the only point of no motion is the fixed end. The air near the fixed end compresses a lot, and the air near the lips moves a lot without compressing. A building in Mode 1 is the same, with lots of compression and expansion in the structure near the ground and lots of motion with no stretching near the top.
  - In **Mode 2**, there are **two points of no motion**, one at the fixed end and one inside the tube.
  - In **Mode 3**, there are **three points of no motion**. Around each of them there is lots of compression and expansion, so those are the springiest places.
- The same reasoning answers questions about a building's higher modes. In **Mode 4** of a building, **the most springiness is at the fixed bottom and at the three points where the building's line crosses its resting position**, four places in all, because those points do not move. The places between them move the most and have the least springiness.
  - Raising the stiffness at a place of maximal movement changes **nothing**, because the structure is not acting like a spring there.
  - Raising the stiffness at a place of no movement makes the frequency **shoot up**, because that spot is very springy.

- The same two-step reasoning answers a question about a string fixed at both ends. Take a small spot where **Mode 3 crosses its middle line** while **Mode 4 is at the top of a bump**, and raise both the mass and the stiffness of that spot.
  - In **Mode 3**, the string does not move at that spot but is stretched the most, so only the extra stiffness counts, and **Mode 3's frequency goes up**.
  - In **Mode 4**, the string moves the most at that spot and hardly stretches, so only the extra mass counts, and **Mode 4's frequency goes down**.
- Adding mass and stiffness at the **very top of a building** lowers the frequency of **every mode**, including Mode 4. The top is the free end, so it has the most motion and almost no springiness in every mode. Only the extra mass counts there, and extra moving mass lowers frequency. **The top of the building corresponds to the lip end of the vocal tract.**

Section 10: Vowel Shapes Locate Constrictions

- A **constriction** is a place where the tongue or lips squeeze the air. Locations are measured from the **glottis**, the tiny space between the vocal folds at the fixed end.
  - In **done**, the central vowel, there is **no constriction**, and the air is pretty uniform along the whole tract.
  - In **dean**, a **front high vowel**, the air is squeezed under the roof of the mouth, about **two thirds of the way from the glottis**.
  - In **Don**, a **low back vowel**, the air is squeezed far back, about **one third of the way from the glottis**.
  - In **dune**, a **back high vowel**, the lips are rounded close together, which squeezes the air **at the lips**.
  - **Dan** is a **front low vowel**.

Section 11: Why a Constriction Can Raise One Formant and Lower Another

- **Compressing air raises its stiffness.** Squeezing air in a tire makes the tire stiffer, so a constriction at a place that experiences lots of compression **raises frequency**.
- **Bunching up air raises its mass**, because squeezing it into a smaller volume raises its **density**. So a constriction at a place where the air moves a lot **lowers frequency**.
- So every constriction **increases both mass and stiffness**, but that does not mean constrictions have no effect on frequency. **The parts of a vibrating medium that move the most alternate with the parts that compress and expand the most.** At any one spot, in any one mode, either the mass or the stiffness is the one that counts.
  - A useful way to answer any constriction question is to ask two things in order. First, is the air at this spot mostly moving or mostly compressing in this mode? Second, which change counts there, the extra mass or the extra stiffness?

Section 12: /u/, /i/ and /a/

- **/u/** rounds the lips, which squeezes the air at the **free end**. The free end moves a lot and hardly compresses in both Mode 1 and Mode 2, so the extra **mass** counts in both. **F1 and F2 both go down.**
  - A speaker who moved from the central vowel into /u/ produced exactly this, with **both formants dropping** in the recording.
- **/i/** squeezes the air about **two thirds of the way from the glottis**.
  - In **Mode 1**, that spot has lots of motion, so the extra mass counts and **F1 goes down**.

    - In **Mode 2**, that spot has lots of compression, so the extra stiffness counts and **F2 goes up**.
    - A speaker who moved from the central vowel into /i/ showed the **first formant dropping a lot** and the **second formant going up**.
    - Put simply, the same squeeze lands on a moving spot for F1 and a springy spot for F2, so one formant falls while the other rises.
- **/a/** squeezes the air about **one third of the way from the glottis**, which reverses the /i/ pattern.
    - In **Mode 1**, that spot near the fixed end compresses a lot, so the extra stiffness counts and **F1 goes up**.
    - In **Mode 2**, that spot moves a lot, so the extra mass counts and **F2 goes down**.
- The reasoning also runs backward. **If a constriction raises a formant a great deal, the air at that spot must be very springy and hardly moving in that mode. If a constriction lowers a formant a great deal, the air at that spot must be moving a lot.**
- A measured **schwa**, made with no constriction, had formants near **500, 1500 and 2500 hertz**, which are the odd multiples a uniform fixed-free tube predicts.

Section 13: The Source Supplies Harmonics and the Filter Shapes Their Heights

- Speech combines **two vibrating systems at the same time**.
    - The **vocal folds** are a fixed-fixed system, so they are the **source**, and they produce **harmonics**. The lowest harmonic is called the **base frequency**, or **F0**, and the others are its multiples. Here the harmonics are **100 hertz apart**.
    - The **air in the vocal tract** is a fixed-free system, so it acts as a **filter**, and its frequencies are the **formants**. For the schwa, the formants sit near **500, 1500, 2500 and 3500 hertz**.
- **The formants shape the heights of the harmonics.** The harmonics are all still there, but **harmonics near a formant are tall, and harmonics far from any formant are short**. The overall outline of the harmonic heights, called the **envelope**, rises into a peak at each formant.
    - **The harmonics stay the same from vowel to vowel, but their heights change, and the envelope is what makes the vowel.**
    - In plain terms, the vocal folds decide which frequencies are present, and the shape of the mouth decides which of them are loud.

Section 14: The /i/ and /a/ Filters

- For **/i/**, the filter keeps **F1 low and F2 high**. The tall harmonics are bunched at the very bottom of the spectrum, then a long stretch of short harmonics follows, and the next tall group appears much higher up. The two lowest formants are **far apart**.
- For **/a/**, the filter **raises F1 and lowers F2**. The first two groups of tall harmonics sit **close together** in the lower part of the spectrum, and the higher formants appear farther up.
- The same source harmonics are under both vowels. Only the filter differs, and moving the constriction is what moves the formants and reshapes which harmonics are loud.

Section 15: A Spectrogram Places Successive Spectra in Time

- **A spectrum describes one instant.** Speech is not one frozen vowel, though. The vocal tract keeps moving, and constrictions are made and unmade, so speech is **time-varying**.

- **A spectrogram stacks many spectra side by side over time.**
  - **Time** runs along the **horizontal axis**.
  - **Frequency**, which was the horizontal axis of each spectrum, becomes the **vertical axis**.
  - **Amplitude**, which was the height of each spike, becomes **darkness**, so the louder a frequency is, the darker it looks.
- The formants appear as **dark bands** that move as the speaker moves. In the sequence **/ia/**, the two lowest bands start **far apart** during /i/, with F1 low and F2 high. As the speaker moves into /a/, **F1 rises and F2 falls**, so the bands **come close together**.
  - Reading a spectrogram means following the dark bands, because those bands are the formants changing over time.

Section 16: A Sentence Over Time

- **Where were you a year ago?** is a good sentence for seeing source and filter working together over time. The vocal folds, as the source, keep producing harmonics, while the vocal tract, as the filter, keeps changing shape, so the envelope over the harmonics changes from moment to moment.
  - On a spectrogram, these changes show up as **dark bands that rise and fall**, since the dark bands are the formants and darkness shows amplitude.
  - Where the vowel sounds like **/i/**, the two lowest bands spread far apart, and during **you**, all the bands drop because the lips are rounded.
- **A spectrum shows one frozen instant, but a spectrogram shows how the formants move**, and real speech is all movement. Reading a sentence like this one means following each band and asking what constriction moved it.

Section 17: Wide Harmonic Spacing Makes Formants Harder to Identify

- Formants can only show up through the harmonics, so the spacing of the harmonics matters. With harmonics every **100 hertz** and formants at **300 and 2100 hertz**, there are **many harmonics** under each formant peak, so the formants clearly **structure the amplitudes** of many harmonics and are easy to see.
- With a higher base frequency, the harmonics are spaced farther apart, such as every **250 hertz**. Fewer harmonics land on each formant peak, and a formant can fall **between two harmonics**, so **it becomes very hard to tell the harmonics from the formants**.
  - A **toddler's** harmonics can be at 400, 800 and 1200 hertz, which is why even listeners can find a toddler's vowels hard to identify unless they know the child.
  - A **soprano's** harmonics can start at 800 hertz and continue at 1600 and 2400, so the words of an opera singer are often very hard to follow.
- **Formants are the smooth envelope, and harmonics are samples of it.** A higher F0 means fewer samples, so the spectrogram stops looking like clean formant bands.
  - A simple way to see it is that the harmonics are like the points used to trace a curve. With many points the curve is obvious, and with only a few points it is hard to see.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 14
Date: Tuesday, October 20
Professor: Khalil Iskarous

Section 1: The Vocal Folds as the First Vibrating System

- Speech uses **two major vibrating systems**. The first is the **vocal folds**, a **membrane** that is **fixed on all sides** and made of a whole bunch of masses and springs. It vibrates in many modes at once, with all the masses going up and down together in Mode 1, half going up while the other half goes down in Mode 2, and more complicated patterns above that. All five modes add together into one combined motion, and this is literally what happens in the throat while a person talks.
  - Because the vocal folds are a fixed-fixed system, their frequencies are **harmonics**, which are whole-number multiples of the lowest one.
  - The **bottom-most harmonic** is called **F0**. Each person has a different F0, but the other harmonics are always its multiples.

Section 2: The Air in the Vocal Tract as the Second Vibrating System

- The second system is the **air in the vocal tract**. It is **fixed at the glottis**, because the closed vocal folds keep the air there from moving back and forth, and it is **free at the lips**, where the air can go in and out. Its modes follow the fixed-free pattern, with **one, three, five and seven quarter-wave packets**, so its frequencies are the **odd multiples** of the lowest one.
- **The closed end is always springy and the open end is always massy, in every mode.** Air can be squeezed against a closed end, but there is nothing to squeeze against at an open end.
  - Near the **closed end**, the slivers of air come very close together and then far apart, so the air **compresses and expands** and acts like a **spring**.
  - Near the **open end**, the slivers stay equally far apart and simply go back and forth, so the air acts like a **mass**.
  - In higher modes, springy places and massy places **alternate** more and more times along the tube.
  - In plain terms, the lips end swings like a heavy door, and the throat end squeezes like a pillow.

Section 3: Reasoning About a Constriction in One Mode

- A constriction can be judged at any spot, in any mode, with the same chain of reasoning. Take a spot inside the tube where, in **Mode 3**, the lines of air are bunching up and spreading apart while hardly moving back and forth. Constricting the tract at that spot raises **F3**.
  - The first step is that constricting air always **increases both its mass and its stiffness**, just as squeezing a tire makes it stiffer.
  - The second step is that at this spot in Mode 3, the air is acting like a **pure spring**. There is compression and expansion but no movement back and forth, so the extra mass does not matter.
  - The third step is that only the extra stiffness counts, and **raising stiffness raises frequency**, so **F3 goes up**.

- Each new spot and mode needs the same steps. The answer is never memorized. It is **built one claim at a time**, with each claim resting on the one before it, which is the same way any good argument is built in science or in art.

Section 4: Vowels Squeeze the Air in Different Places

- Images taken while people speak show where each vowel squeezes the air in the tube that runs from the closed glottis to the open lips.
  - The vowel in **done** has **no constriction** at all.
  - **dean** squeezes the air near the front, about **two thirds of the way from the glottis**.
  - **Don** squeezes the air far back in the throat, about **one third of the way from the glottis**.
  - **dune** squeezes the air **at the lips** by rounding them.
  - **Dan** is a low front vowel.
- In the mass-spring model of the tube, a constriction is drawn as **heavier masses and stiffer springs** at that spot. It always raises both mass and stiffness, but because moving places and springy places alternate, **its effect depends on what the air is doing there in each mode**.

Section 5: /u/, /i/ and /a/

- **/u/** squeezes the air at the lips. At the open end the air goes back and forth with hardly any compression in **every** mode, so the air there is a pure mass. The extra stiffness does not matter, the extra mass does, and **F1 and F2 both go down**.
  - Compared with schwa's reference values of about **500 and 1500 hertz**, a measured **/u/** had an F1 down in the low two hundreds, well below 500.
- **/i/** squeezes the air about two thirds of the way from the glottis.
  - In **Mode 1**, the air there moves back and forth, so it acts like a mass, the extra mass counts, and **F1 goes down**.
  - In **Mode 2**, the air there compresses and expands, so it acts like a spring, the extra stiffness counts, and **F2 goes up**.
  - A speaker moving from schwa into /i/ showed exactly this, with **F1 going down and F2 going up**.
- **/a/** squeezes the air about one third of the way from the glottis.
  - In **Mode 1**, that spot is close enough to the closed end that the air is mostly springy, so the extra stiffness counts and **F1 goes up**.
  - In **Mode 2**, that spot sits between two springy regions, so the air there is mostly moving, the extra mass counts, and **F2 goes down**.
  - A speaker producing /a/ showed **F1 going up and F2 going down**, and a typical /a/ has its F1 and F2 fairly close together.
- **Whenever stiffness is what matters at a spot, the frequency goes up, and whenever mass is what matters, the frequency goes down.**

Section 6: The Source Sets the Harmonic Spacing

- The vocal folds are the **source**, and their mass and stiffness set **F0** and therefore the spacing of the harmonics.
  - **Big, heavy, loose vocal folds** have a large mass and small stiffness, so their **F0 is low**, around 100 hertz, with harmonics at 200, 300 and so on.

    - **Tighter, lighter vocal folds** have a higher F0, such as 250 hertz, with harmonics at 500 and 750.
    - **Very tight, light vocal folds** have an even higher F0, such as 350 hertz, with harmonics at 700 and 1050.
- **F0 is not fixed while people talk.** Only a robot keeps F0 constant. Speakers keep changing F0 by tightening and loosening the vocal folds with the muscles in the larynx, and listeners use those changes to get meaning.
    - **The dog slept here** said as a statement has F0 falling at the very end, while the same words as a question have F0 rising at the end.
    - The way F0 moves also tells listeners whether a person is being sarcastic, pretending, or even lying.
- F0 can be measured in **Praat** by taking a little piece of speech, viewing a spectral slice and picking the lowest harmonic. One measured voice had F0 around 170 hertz, and the next harmonic sat at about twice that value, as the theory predicts.

Section 7: The Filter Selects the Formants

- The air in the vocal tract is the **filter**, and its formants depend on **where the constriction is**. Typical values show the patterns.
    - **Schwa** has F1 near **500** and F2 near **1500** hertz.
    - **/u/** has F1 near 300, F2 near 1000 and F3 near 2000, so all of them are lower.
    - **/i/** has F1 near 300 and F2 near 2200, which are very far apart.
    - **/a/** has F1 and F2 both near 800 to 900 hertz, which are very close together.
- **The core rule is that a formant encourages the closest harmonics to be audible, and harmonics far from that formant are muted.** If there are harmonics near 500 hertz during a schwa, they are amplified, and the farther a harmonic is from 500, the weaker it comes out. Between two formants the harmonics get weaker, then they grow again near the next formant.
    - **The vowel shape is the filter. The voiced source supplies the harmonics, and the filter chooses which harmonics survive.**

Section 8: One Filter Can Look Different at Different Pitches

- **The lower F0 is, the more harmonics the formants can shape.** With heavy, loose vocal folds and harmonics every 100 hertz, many harmonics sit under each formant, so the shaped spectrum traces the formant peaks smoothly and is easy to read.
- With very light, tight vocal folds, the harmonics are far apart, and a formant at 500 hertz may have **no harmonic near it to amplify**. Sparse harmonics can miss a formant or make it look ambiguous.
    - An **opera singer** is an extreme case. Their harmonics are so far apart that the formants do not line up with any harmonic, so listeners struggle to hear which vowel is being sung. Singers are even trained to shape the mouth so the formants line up with the harmonics.
    - A **small child** of two or three has an F0 near 500 hertz, so very few harmonics fall near the formants. Unless a listener is one of the child's caretakers and is used to the child's speech, it is very hard to tell what the child is saying.
    - A simple way to see it is that the formants are the same in every case, but a high voice gives the listener only a few harmonics to hear them through.

Section 9: Spectrograms of Real Speech

- A **spectrogram** shows how speech changes over time. **Time** runs along the horizontal axis, **frequency** runs up the vertical axis, and **darkness** shows how much energy, or amplitude, there is.
- In a recording of **Where were you a year ago?**, the formants never sit still.
  - Where the vowel sounds like **/i/**, F2 rises high and F1 stays low, because the air is squeezed where Mode 2 is springy and Mode 1 is massy.
  - During **you**, the formants drop, since rounding the lips lowers all of them.
  - Toward the end of the sentence, F2 goes up and then down again as the vowels change. **All the formants are constantly changing during speech.**
- **All the formants are constantly changing during speech, so speech is not a string of frozen vowels.** Every rise and fall of a formant can be predicted with the same mass and spring reasoning, by asking where the constriction is and whether the air there is moving or compressing in each mode.
  - This constant change is why a network that sees two formants from one frozen moment cannot understand real speech, and why speech recognition needs systems that take in whole stretches of sound over time.
  - In plain terms, a spectrogram of a real sentence is a moving record of the tongue and lips, and reading it means following the dark bands as they rise and fall.

Section 10: Speech Perception Integrates Over Time

- A spliced recording reveals how complex speech perception is. A speaker says **ma** and **na**. The **m** part of ma is cut out and joined to the **a** part of na. Since the consonant portion came from ma, the result should sound like ma, but **listeners clearly hear na**.
- **The perception of m or n comes more from the vowel than from the nasal sound itself.** The formants of the vowel differ depending on the consonant before it.
  - After **m**, one formant of the vowel stays fairly **stationary**.
  - After **n**, that formant shows a **big drop** at the start of the vowel.
  - So the vowel portion carries the information about which consonant was heard, and listeners decide what consonant they heard **during the vowel**, after the consonant is already over.
- This was one of the first striking results in speech perception, published in the **1950s** by the linguist **André Malécot**. It means listeners do not decide on one slice of sound, then the next, then the next. They **take in whole stretches of speech and then decide** about sounds that came first by using what follows them.
  - For about seventy years, speech recognition machines that treated each slice as its own sound **failed**, despite billions of dollars of effort.
  - A machine that understands speech has to be complex enough to use future sound to decide about the present, which is why modern speech recognition relies on large language models built from many neural networks.

Section 11: Formants as Input to a Vowel Recognizer

- A **neural network** can be trained to recognize vowels from formants. Its **two inputs** are **F1 and F2**, measured from real recordings of five vowels, and its job is to decide which vowel was spoken. **The knowledge of the network is in its weights**, which are learned from the data.

- The network has **five output neurons**, one for each word, **done, dune, dean, Don and Dan**. Between the input and the output it has **three hidden layers**, with **3, 2 and 4 neurons**.
  - Information arrives at the input, a little like sound arriving at the ear.
  - Each hidden layer turns the numbers it receives into a new representation, a little like successive stages of processing in the brain.
  - The output layer gives the network's decision about which vowel it heard.
- Every neuron in one layer connects to every neuron in the next, so the network is **fully connected**. This version is **purely linear**, meaning it has **no activation functions**, so each neuron just passes on the result of its inner product.

Section 12: Random Weights Give Signed Contributions

- Before training, **every weight is a random number between minus one and one**. A **positive** weight makes its input push the next neuron up, and a **negative** weight makes its input push the next neuron down.
- The example input is **F1 of 320 and F2 of 2000 hertz**. A very low F1 with a very high F2 is the pattern of **/i/**, so the correct answer is **dean**.

Section 13: Forward Flow and the Output Distribution

- **Forward propagation** carries the input through the network one layer at a time. Each neuron computes an **inner product**, which means multiplying each incoming number by its weight and adding the results.
  - For the top neuron of the first hidden layer, F1 is multiplied by its weight, F2 is multiplied by its weight, and the two products are added to give that neuron's activation.
  - The other neurons do the same with their own weights, so the first hidden layer produces three numbers. Those numbers pass through the next weights to make two numbers, then four, then five at the output.
  - None of these calculations ever needs to be done by hand. What matters is that every step is an inner product.
- The output is a **probability distribution** over the five words, with values of **.5, .1, .1, .1 and .2**, which add up to one. The network puts its highest probability on **done**, so with random weights it is a **stupid network** that gets the answer wrong.
  - The correct answer is given as the target distribution **0, 0, 1, 0, 0**, with all of the probability on dean.
  - **Stochastic gradient descent** then changes each weight, so that the next time the network sees 320 and 2000, the inner products lead to a high probability for dean.
  - In plain terms, the network starts out guessing, the target says what the right guess was, and learning nudges every weight so the right guess wins next time.

Section 14: The Same Network in PyTorch

- **PyTorch** declares the network as a list of layers.

```
import torch
import torch.nn as nn

net = nn.Sequential(
    nn.Linear(2, 3, bias=False),
    nn.Linear(3, 2, bias=False),
    nn.Linear(2, 4, bias=False),
```

```
    nn.Linear(4, 5, bias=False),
)
```

- **Sequential** means the network has **one layer after another, in a sequence**. **Linear** means **do inner products** of weights and previous activations.
  - **nn.Linear(2, 3)** reads as **take input from 2 neurons, and have 3 neurons**. The next line takes input from those 3 and has 2, then 2 in and 4 out, then 4 in and 5 out.
  - Turning off the **bias**, by setting it to False, keeps every layer purely linear, matching the diagram of the network. This code only declares the network and does not train it.
- **Only layers that do calculations get a line.** The **input layer** has no weights coming into it and does no inner products, because it just takes in numbers. So the input layer never gets its own line, and the first line describes the **first hidden layer**.
  - This network has **five layers in total**, which are the input layer, three hidden layers and the output layer.
  - It has **three hidden layers**, and the **last line is always the output layer**.
  - "How many layers" and "how many hidden layers" are different questions, so the question has to be read carefully.
- Changing the network means changing the numbers in the lines. To **add a hidden layer of 3 neurons right before the output**, the first three lines stay the same. The old last line, 4 in and 5 out, becomes **4 in and 3 out**, and a new last line takes **3 in and gives 5 out**. Read aloud, the network becomes 2 in and 3 out, 3 in and 2 out, 2 in and 4 out, 4 in and 3 out, and 3 in and 5 out.
  - A quick check for any rewrite is that each line's first number must match the previous line's second number, because each layer takes its input from the layer before it.
- The same reading works for any network written this way. A network declared as 4 in and 3 out, 3 in and 8 out, 8 in and 4 out, 4 in and 4 out, and 4 in and 5 out takes **4 numbers as input**, has **four hidden layers** with 3, 8, 4 and 4 neurons, and decides among **5 categories**, one for each output neuron.
  - Drawn as a network, it has a column of **4 input neurons**, then hidden columns of **3, 8, 4 and 4 neurons**, then a column of **5 output neurons**, with every neuron connected to every neuron in the next column.
  - Asked what the target distribution looks like when the first category is correct, the answer puts a **1 in the first slot and 0 everywhere else**.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 15
Date: Tuesday, October 27
Professor: Khalil Iskarous

Section 1: Two Formants Feed a Fully Connected Network

- A **neural network** can act as a very simple **speech recognizer**. It takes the **two formants, F1 and F2**, of a vowel and decides which of **five words** it heard, **done, dune, dean, Don or Dan**. The networks studied here are the kind used fifteen or twenty years ago, and modern **large language models** use networks like these as their **atoms**, building them into much bigger structures.
- The network has **2 input neurons**, **three hidden layers** with **3, 2 and 4 neurons**, and **5 output neurons**. Every neuron connects to every neuron in the next layer, so it is **fully connected**.
  - The **input neurons** just take numbers in from the environment, such as one speaker's F1 of 400 and F2 of 1900 for some vowel.
  - **The knowledge of the network is in its weights.** Each connection has a weight, which is a positive or negative number, and the whole network is nothing more than a machine for processing numbers with those weights.

Section 2: Forward Propagation Combines Weighted Inputs

- **Forward propagation** carries one input through the network to a decision. The example input is **F1 of 320 and F2 of 2000**, which has the pattern of **dean**.
  - Each neuron computes an **inner product**. For the top neuron of the first hidden layer, the weight on F1 is multiplied by 320, the weight on F2 is multiplied by 2000, and the two products are added.
  - The result of that inner product, with no other change, is called the neuron's **linear activation**, and here it comes out slightly below zero.
  - The other neurons do the same with their own weights. Their activations then become the inputs to the next layer, which does more inner products, and so on until the output layer.
- At the end, the output numbers are turned into a **probability distribution**, a set of numbers between zero and one that **add up to one** and stand for the network's **decision**. The operation that does this is called **softmax**, and all that matters about it is that it converts numbers into probabilities, with a bigger number giving a bigger probability. Probabilities and percentages are the same idea, since a probability of 0.55 is a 55 percent chance.
  - Here the network gives **done about 56 percent**, and much less to each of the other words. So the network says, in effect, **"I heard done,"** even though the input was dean.
- **Knowledge has been reduced to positive and negative numbers, and inference has been reduced to multiplying and adding**, which are operations from second grade. **Inference** means working out a decision, and the systems people chat with every day make their inferences with nothing but these elementary operations.
  - This is less surprising than it seems, because brains are chemical machines. The chemistry of ions such as sodium, potassium and calcium inside neurons performs operations close to adding and multiplying.

Section 3: Raising an Activation or a Weight

- Because every step is an inner product, the effect of raising one number can be worked out without any arithmetic. Consider the top neuron of the last hidden layer, whose activation is about 1.57, and its weight of positive 0.1 into the **done** output.
  - If that **activation goes up**, its contribution to the done output, which is the activation times a positive weight, goes up, so the **probability of done increases**.
  - If that **weight goes up**, from 0.1 to 0.2, the contribution of the positive activation also grows, so the **probability of done increases** again.
  - **A positive number times a positive number is positive.** So raising either one of them makes that neuron push the output up more.
- The questions here are about understanding what happens, never about doing the multiplication. A good answer names the direction, up or down, and then the reason, such as "the probability of done goes up because the weight and the activation are both positive, so their product adds more to the done output."
  - In plain terms, each neuron votes for each output through its weight, and a positive vote from an active neuron always helps that output.

Section 4: Backpropagation Uses the Difference Between Prediction and Target

- **Every network starts out stupid**, because its weights start out **random**. It becomes smarter only by being told the right answer.
  - The right answer is given as a **desired output**. For a recording of dean, the desired output is **0, 0, 1, 0, 0**, which says there is a 100 percent chance it was dean and zero for everything else. This kind of list, with a single one and zeros everywhere else, is called a **one-hot encoding**.
  - The output the network actually produced is the **computed output**, and the gap between the computed output and the desired output is the network's **error**.
  - The desired outputs come from real data, such as a table of each student's formants together with the vowel that student actually said.
- **Backpropagation** is the algorithm that changes the weights so the network makes better decisions. If the same 320 and 2000 were given again after learning, the probability of done should go down and the probability of dean should go up.
  - **Learning happens gradually, one step at a time.** Just as with stochastic gradient descent, the weights do not jump to the right values in one step. They are nudged a little, then the next piece of data nudges them again, and so on.
  - Each weight moves a small step in whichever direction makes the error smaller, and the **learning rate** sets how big that step is.
  - The weights are updated **starting from the end**. First the weights into the output change, then the weights into the third hidden layer, then the second, then the first. The activations from the forward pass stay as they were while this backward pass happens.
- This is why AI needs huge **data centers**. A modern chatbot can have **trillions of weights**, and backpropagation has to nudge all of them, over and over, for months.

Section 5: Predicting How a Weight Changes the Error

- For the **last weight**, the one of 0.1 that links the top neuron of the last hidden layer to **done**, the effect is easy to see.

    - That weight multiplies an activation of about 1.57, so raising the weight would raise the probability of done.
    - The desired probability of done is **zero**, so raising the weight would make the error **worse**.
    - So that weight should be **lowered**, which shrinks its contribution and pulls the probability of done down toward zero. The decision takes only one inner product to reason out.
- For the **first weight**, the one of 0.005 near the input, the effect is very hard to see.
    - Changing it changes one activation in the first hidden layer. That changes the activations in the second hidden layer, which change the third, which finally change the output probabilities.
    - **Its effect on the decision is mediated by all the intermediate calculations**, and it also depends on all the later weights. So just by looking at it, there is no way to tell whether it should go up or down. Backpropagation still works it out.

Section 6: Why Backpropagation Starts at the End

- **The last weights directly determine the output probabilities**, so the error is very directly determined by them, and the difference between the computed and the desired output can be used right away to improve them.
- **The first weights affect their activations, which affect two more layers of activations along with other weights.** It is really hard to figure out how much an early weight is to blame for a mistake made all the way at the end. Its effect on the error also **includes the effect of the last weights**, since the last weights are downstream of it.
- So **improving the last weights is the easy problem**, and improving the first weights is a much bigger problem that already contains the easy one. The method for solving a series of problems like this, where small problems are solved first so they can be used for bigger problems that depend on them, is **dynamic programming**, developed by **Bellman** and **Viterbi**.
    - **Backpropagation is stochastic gradient descent plus dynamic programming.** Stochastic gradient descent changes the numbers gradually, and dynamic programming supplies the **back** in backpropagation.
    - Backpropagation was worked out in the **mid-1980s**, and in 2024 a Nobel Prize recognized it, because there would be no AI without it.
    - Research over the last ten years suggests that single neurons in the brain also work in a forward-then-backward way, using their chemistry, so this clean two-phase process may not be so different from what brains do.
    - The short version of the exam answer is that the last weight's effect on the decision is one simple step, while the first weight's effect goes through many layers and depends on the later weights, so the easy problem is solved first and its answer is reused for the harder ones.

Section 7: Pairing Each Recording with Its Target

- Training needs two tables of numbers, which are called **matrices**. Each row pairs **one recording** with **the vowel it really was**.
    - The **input matrix** has one row per recording and **two columns**, F1 and F2, because the network takes two numbers in. With about 110 recordings, it is 110 by 2.

- The **output matrix** has one row per recording and **five columns**, one for each word, with a **1 marking the true vowel** and 0 in the other columns. It is 110 by 5.
- So the network is told, row by row, that if it is given formants like 800 and 1700, the desired answer is one particular vowel, and if it is given 700 and 1100, the desired answer is another.

Section 8: One Training Step in PyTorch

- A short **Python** program, using the **PyTorch** library, builds and trains this network.

```
import torch
from torch import nn

X = torch.load("Formant_Inputs.pt").float()
Y = torch.load("TrueVowel_Outputs.pt").float()

net = nn.Sequential(
    nn.Linear(2, 3),
    nn.Linear(3, 2),
    nn.Linear(2, 4),
    nn.Linear(4, 5)
)

sgd = torch.optim.SGD(net.parameters(), lr=0.1)
sgd.zero_grad()
loss = nn.CrossEntropyLoss()(net(X), Y)
loss.backward()
sgd.step()
```

- The **import** lines tell the program to use the **torch library**, which does all the background work so that the network can be described in just a few lines. The two **load** lines bring in the two matrices, the formants as X and the true vowels as Y.
- **Sequential** means the network is a **sequence of layers**, one after another. **Linear** means **do inner products**, since all a linear layer does is multiply and add.
  - **nn.Linear(2, 3)** describes the **first hidden layer**, which takes **2 inputs and provides 3 outputs**. The next lines take 3 and give 2, take 2 and give 4, and take 4 and give 5.
  - The network has **five layers**, but there are only **four lines**, because the **input layer does no processing**. It just receives numbers, so its only trace in the code is the 2 in the first line.
- The last lines carry out **one training step** using all 110 rows at once. **SGD** is stochastic gradient descent, and **lr** is the **learning rate**, which sets the size of each step. The **loss** measures the network's mistakes, comparing its outputs with the true vowels, and **backward** runs backpropagation to work out how each weight should change. **step** then actually changes the weights.
  - In plain terms, the program describes the network, shows it the data, measures how wrong it is, works backward to find who is to blame, and then adjusts the weights a little.

Section 9: The Sigmoid Activation Function

- **Lord Edgar Adrian** won the **Nobel Prize in Physiology or Medicine in 1932** for his discovery that **neurons are nearly digital**. A real neuron is roughly **on or off**, either firing or not firing, instead of taking any value at all. The neurons in the network so far are not like that, because their activations are any real numbers, such as slightly below zero, about 6.6 or about 2.7.
- Knowing this discovery, people added an **activation function** called the **sigmoid** to neural networks. The sigmoid takes the inner product and squashes it.
  - If the inner product is a **large negative number**, the sigmoid gives about **0**, so the neuron is **off**. An inner product of minus 4 gives almost exactly zero, and minus 10, minus 100 or minus 1000 all give about zero too.
  - If the inner product is a **large positive number**, the sigmoid gives about **1**, so the neuron is **on**. An inner product of 4 gives almost exactly one, and 6 or 100 also give about one.
  - Only inner products **near zero** get values in between, so the sigmoid **digitizes** the neuron, much like the chemistry that keeps real neurons from firing without limit.
- The sigmoid also matches something deep about **perception** in animals and humans. **Some things that are very different are perceived as the same, and some things that are very similar are perceived as different.**
  - A friend is recognized as the same person whether they wear a hat or not, or a different shirt on a different day. Perception ignores many real differences, just as the sigmoid gives minus 1000, minus 100 and minus 20 all the same value.
  - Two universities can be close to each other, both large, and alike in their labs and their football, yet USC and UCLA are perceived as completely different. Small differences can matter enormously, just as two inputs just below and just above zero land on opposite sides of the sigmoid's jump.
  - So the sigmoid lets the same difference in input count for nothing in one place and count for everything in another. In plain terms, the flat parts of the curve lump different things together, and the steep middle splits nearly identical things apart.

Section 10: Applying the Sigmoid One Neuron at a Time

- Adding the sigmoid to each hidden neuron changes every number downstream. The inputs stay as formants, but each hidden neuron now passes on its sigmoid value instead of its raw inner product.
  - The first hidden neuron's inner product, slightly below zero, becomes about **0.4**.
  - The second, about 6.6, becomes about **1**, so that neuron is fully on.
  - As each neuron is changed, the activations in the later layers are recalculated, and the output probabilities shift.
- With the same random weights and the sigmoid on all nine hidden neurons, the decision moves away from done. The network now spreads its probability more evenly and gives its highest probability to **Dan**, around one third, so it is **still wrong** about dean. The sigmoid changes how the network computes, but only training with backpropagation can give it good weights.

Section 11: The Same Program with the Sigmoid

- The program changes in only one way. **nn.Sigmoid** is inserted after each **hidden** linear layer, which means "apply the sigmoid," or make that layer digital.

```
net = nn.Sequential(
    nn.Linear(2, 3), nn.Sigmoid(),
    nn.Linear(3, 2), nn.Sigmoid(),
    nn.Linear(2, 4), nn.Sigmoid(),
    nn.Linear(4, 5)
)
```

- The **last layer has no sigmoid**, because its outputs go straight into the loss calculation, which turns them into probabilities with softmax. The rest of the training program, with SGD at a learning rate of 0.1, stays the same.
- Whether adding the sigmoid makes the vowel recognizer better or worse is something to test by training both versions. Other activation functions that look somewhat like the sigmoid, such as one called **ReLU**, can be swapped in to see whether they improve recognition. **Many of the most basic ideas in AI come from how real brains work**, such as neurons, inner products that gather input from many neurons, and the near-digital firing captured by the sigmoid.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 16
Date: Thursday, October 29
Professor: Khalil Iskarous

Section 1: A Network That Recognizes Vowels

- This **neural network** simulates a very simple piece of human perception. It takes the **two formants**, F1 and F2, of an isolated vowel and decides which of five words it heard, **done, dune, dean, Don or Dan**. It has 2 input neurons, hidden layers of 3, 2 and 4 neurons, and 5 output neurons, and every neuron connects to every neuron in the next layer.
- **The weights are the network's knowledge.** Each one is a positive or negative number, and the network is told the right answer for each recording as a probability distribution. For a speaker's /i/, it is told **0, 0, 1, 0, 0**, which means a 100 percent chance that the word was dean.
  - At first **the weights are random**, so the network knows nothing, and the distribution it produces is usually very different from the right one.
  - The difference is a **mistake**, and the network learns by correcting its mistakes.

Section 2: Forward Propagation and the Network's Decision

- **Forward propagation** carries the input through the network. With **F1 of 320 and F2 of 2000**, one speaker's /i/, each neuron in the first hidden layer computes an **inner product**, multiplying each formant by its weight and adding the results. Those activations feed the next layer, which does its own inner products, and so on until the output gives a probability for each word.
  - With random weights, the network gives its highest probability, more than half, to **done**. So it decides it heard done, even though the right answer is dean.
- To fix the mistake, **all the weights must change**, the ones near the output and the ones near the input. The algorithm that does this is **backpropagation**, which is made of **stochastic gradient descent** and **dynamic programming**.
  - **Stochastic gradient descent** is the same algorithm used to improve an average from data, now used to improve every weight. It has the same **learning rate**, with the same tradeoff between learning fast and learning accurately.
  - **Dynamic programming** supplies the order. The weights at the end are improved first, then the ones before them, and so on back to the beginning, so the most trivial problem is solved first and the hardest problem last.

Section 3: Why the Last Weights Are the Easiest to Change

- **Changing a weight at the very end is directly and immediately reflected in the output probabilities**, which are the network's decision. Those weights are exactly the ones used in the final inner products.
  - For example, the probability of done comes partly from a weight of 0.1 times an activation of about 1.57.
  - The network said done when the answer was dean, so the probability of done is **too high**. Raising that 0.1 to 0.5 would raise the probability of done, which is the exact opposite of what is wanted, so that weight should go **down**.
  - For the last weights, it is easy to tell whether each should go up or down.

- **Changing a weight near the input sets off a chain of changes.** It changes the activations of the first hidden layer, which change the second hidden layer, which change the third, which finally change the decision. So deciding how to change an early weight is a very hard problem, as hard as the dynamic programming puzzle where simply picking the smaller next step gives the wrong answer.
- **Backpropagation changes the last weights first, then the ones before them, and so on to the first.** Each later problem is easier because something has already been learned from the problems solved before them.
  - The error itself says which way each output should move. Done at about 56 percent should go down, dean should go up, and the others should go down.
  - Modern systems apply this same procedure to about **six trillion weights**, which is what the huge data centers are doing.
  - **Geoffrey Hinton** won the **2024 Nobel Prize in Physics** for backpropagation, an idea at the center of much of today's economy, and it has roots in physics through dynamical systems.
  - In plain terms, the blame for a mistake is easiest to assign to the weights closest to the mistake, and that blame is then passed backward to the weights that came before.

Section 4: Real Vowel Data Overlaps

- The real formant data from the class shows why AI is hard. Each recording is a dot, with **F1 along the horizontal axis** and **F2 up the vertical axis**, and there are 525 dots, 105 for each vowel.
  - The data do show a pattern. The **/i/ dots sit high**, with a **high F2 and a low F1**, just as the theory predicts, and /i/ is clearly different from /u/.
  - But there is **enormous overlap**. Many of the /æ/ dots sit among the /i/ dots, many overlap with schwa, and the /u/, /a/ and schwa clouds pile on top of one another.
  - Some dots are probably mistakes in measurement. Real data always has errors, just as speech to a phone on a noisy train does, so they were left in.
- **If the vowels sat in separate clusters, AI would be easy.** Because they overlap, a network has to see the data many times to learn where each vowel is most likely, and even then many points are ambiguous. For a new speaker whose formants land in the middle of the overlap, the best answer is only the vowel whose dots are most common there.
  - Trying to remove the differences between speakers, which is called **normalization**, was attempted for about seventy years without success. Hearing a lot of speech from the same speaker does help, because it gives the system more information about that person.
  - This overlap was first discovered in the early 1950s by **Peterson and Barney**, once spectrograms, which had been kept secret by the militaries during the Second World War, became available. Good speech recognition only arrived around 2011 or 2012, with neural networks trained by backpropagation.
  - Surprisingly, the dynamic problem of speech changing over time turned out to be easier for modern systems than this static one, because large language models use information from the left and the right of each moment.

Section 5: Learning Happens Only Through Errors

- Training uses **two matrices** of numbers. The first has one row per recording with **F1 and F2**, and the second has the matching row of **zeros and a single one** that says which vowel it really was. Learning picks a row, computes the network's decision, compares it with the true answer, and changes the weights if the decision is wrong.
- **If the network makes the right decision, there is no error, so there is nothing to learn.** This can be a real problem. A network can keep getting the right answer by **memorizing** its data without learning the pattern underneath it, and stopping its learning then would be stopping too early.
- Children learning English show the same thing, as **Carol Chomsky** observed in the early 1960s.
  - Very young children first say **go** and **went** correctly, because they are simply **memorizing** what they hear.
  - Then they notice the real pattern, that most English verbs form the past by adding a sound at the end, as in **jump** and **jumped**, and they start saying **goed**.
  - Only later do they learn that some verbs are **exceptions**, and they get both the regular verbs and the exceptions right.
  - In plain terms, getting the right answer early can come from memory, and real learning sometimes shows up as new mistakes.

Section 6: Reading the PyTorch Program

- The program that trains the network is short, and the exam can ask what its lines mean, though never how to write it from scratch.

```
import torch
from torch import nn

net = nn.Sequential(
    nn.Linear(2, 3),
    nn.Linear(3, 2),
    nn.Linear(2, 4),
    nn.Linear(4, 5)
)

sgd = torch.optim.SGD(net.parameters(), lr=0.1)
loss = nn.CrossEntropyLoss()(net(X), Y)
loss.backward()
sgd.step()
```

- **The import lines bring into Python the programs for doing inner products, backpropagation and related work**, so none of that has to be written by hand.
  - This works because of **libraries**, collections of programs written by one group that others can use without knowing how they work inside. The idea of software built this way goes back to **Margaret Hamilton's** work on the software that flew **Apollo 11** to the moon.
  - Around 2015, **Facebook** released **PyTorch** and **Google** released **TensorFlow**, so that researchers no longer had to write their own backpropagation code. Most people now use PyTorch, often just called torch.
  - The **load** lines read the formant matrix and the matching vowel matrix from files on the computer.

- **Sequential means a network that is a sequence of layers**, one after another, which describes every network covered so far. A large language model is an example of a network that is not just a sequence of layers.
  - Each **Linear** line is one **processing layer**, which has weights and does inner products with the information coming in. The input layer does no processing, so the five layers need only **four lines**.
  - **nn.Linear(2, 3)** says "I receive information from 2 neurons, and I have 3 neurons." The other lines read the same way.
- The last lines are all that PyTorch needs for learning.
  - **SGD** is stochastic gradient descent, and **lr** is the **learning rate**, set here to 0.1. **A higher learning rate learns much faster but less accurately, and a lower learning rate learns more accurately but much more slowly.**
  - So changing the line from a learning rate of 0.1 to 0.3 makes the network **learn faster**, but its **accuracy goes down**.
  - **Loss** is the **mistake**, and the program calculates the mistakes so it can learn from them.
  - **backward** is **backpropagation**, and **step** updates the weights.

Section 7: The Sigmoid Makes Neurons Nearly Digital

- Artificial neural networks borrow their basic ideas from discoveries about real brains. There are **neurons**, and each neuron **gathers information from many other neurons**, which is what the inner product does.
- **Lord Edgar Adrian**, who won the **Nobel Prize in 1932** for work in the 1920s, discovered that **neurons are nearly digital**. A real neuron is either **firing** or **not firing**, not at some level like 2, 16 or 34. The neurons in the network so far are not like that, since their activations are real numbers like slightly below zero, 6.64, 2.72 and 1.57.
- The **sigmoid** is an **activation function** added to the network to work like Adrian's discovery. It takes an inner product and gives back a number between zero and one.
  - An inner product of **minus 4**, minus 6 or minus 1000 gives about **zero**, so the neuron is **off**.
  - An inner product of **2** gives about 0.8 or 0.9, and **4**, 6 or 100 give about **one**, so the neuron is **on**.
  - So the sigmoid **digitizes** neural processing, the way brain chemistry does. Without that limit, an activation of a thousand would be like lightning coming out of someone's head.
- The sigmoid also reflects perception. **Some things that seem alike are perceived as different, and some things that are very different are perceived as the same.** Very different large negative inputs all come out as off, while two nearly equal inputs on either side of the steep middle come out as off and on.

Section 8: Applying the Sigmoid in the Network

- Applying the sigmoid one hidden neuron at a time changes the whole calculation downstream. Each neuron's raw inner product is replaced by its sigmoid value, every later activation is recalculated, and the output probabilities shift.
  - The first hidden neuron, slightly below zero, becomes about 0.4, and the second, about 6.6, becomes nearly 1.
  - With the sigmoid on all nine hidden neurons and the same random weights, the network's choice moves from done to Dan, so it is still wrong until it is trained.

- In the program, **nn.Sigmoid** is added after each hidden linear layer, which means "apply the sigmoid," or make that layer digital. The last layer gets no sigmoid, because its outputs go into the loss calculation.
  - The sigmoid can also be replaced by other functions that look somewhat like it, such as **ReLU**, and trying them out shows whether they improve recognition or make it worse.
  - **AI is duplicating abilities that animals have.** Its most fundamental ideas come from biologists' discoveries about how brains work, and computer scientists then add ideas of their own on top.

Section 9: Training the Network from One Starting Point

- The full training program runs **2000 updates**, each one using all the recordings, with a learning rate of 0.1. The formants are divided by a thousand first, turning hertz into kilohertz, and after every update the program records the **accuracy**, which is the percent of recordings the network classifies correctly. The word with the largest score counts as the network's answer.
- **The accuracy starts at 20 percent.** With random weights the network knows nothing, but with **five vowels**, even a random guess is right **one time out of five**. That level is called **chance**.
  - With **ten categories**, chance would be **10 percent**, since a random guess would be right one time in ten.
- After a while the accuracy **jumps up to about 40 percent**, so the network is clearly learning something. Then it **levels off**, finishing near **45 percent**. Each pass of learning is called an **epoch**, and it takes thousands of them for a problem like this.

Section 10: Ten Starting Points, the Same Final Accuracy

- Training was repeated **ten times** on the same data, each time starting from a **different set of random weights**. A starting set of random numbers is called a **seed**, though the word itself will not be asked.
  - The runs take **different paths**. One stays at chance for about 500 epochs before jumping up, another jumps up very early, and another climbs more gradually.
  - **All ten end at the same accuracy**, close to 45 percent. Learning by stochastic gradient descent always involves chance along the way, but the end point is the same.
- Children learning language are similar. One child's home might talk mostly about sports and another's about politics, so each child hears very different words and voices. Yet by about age four, they speak the same language, because the fundamental patterns underneath their different inputs are the same.
- **The network levels off near 40 percent, not 100, because the data overlap so much.** For a point in the middle of the overlap, even a person looking carefully at the data would have to squint and guess. AI is not magic, so a problem that is very hard for humans is hard for machines too.
  - Adding or removing layers, or changing the number of neurons in each layer, is the next experiment, to see whether those changes improve on the 40 percent or make it worse.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 17
Date: Tuesday, November 3
Professor: Khalil Iskarous

Section 1: Neural Networks from 1943 to 1986

- A **neural network** takes in information about one moment, uses its **weights** to process it and makes a decision, and it learns those weights by making errors and correcting them through **backpropagation**. The idea began in **1943** with a paper by **McCulloch and Pitts**, which proposed that simple mathematics could model how the brain works.
  - The paper starts from the **all-or-none** character of nervous activity, where all means active and none means inactive. That is the same idea as the **sigmoid**, so the very first paper on artificial neural networks starts from biology about how real neurons behave.
  - Warren McCulloch was a **psychiatrist**, and the earliest work on neural networks aimed to model how the mind works, including mental illness, not to make money.
- By **1986**, the ideas had matured into the two **Parallel Distributed Processing** books, written by a group at UC San Diego. **Parallel** means many neurons compute their activations at the same time, and **distributed** means the information from the inputs is spread across all the neurons. Forward propagation and backpropagation were well formed by then, and one contributor, **Geoffrey Hinton**, later won the **2024 Nobel Prize in Physics** for this work.
- **Given some static vector of formants, these networks could make a decision about a few possible categories.** A **vector** is just a list of numbers, and static means one fixed list from one moment. But **human cognition is far more complex** than taking in one list of numbers and giving one decision at the end.
  - **Speech is incredibly time-varying**, because every little moment in time gives a new set of formants.
  - **Language is time-varying too**, since a sentence is a whole bunch of words, one after another.
  - So giving a network one bunch of numbers can never model cognition that unfolds in time.

Section 2: Jordan's Recurrent Network for Speech Production

- In **1985**, **Michael Jordan** developed **recurrent neural networks** to make sense of **speech production**, meaning how the tongue, the velum and the lips move over time. Features such as nasality, backness and lip rounding each switch on and off at different moments across a sequence of sounds, and he realized that a regular network could never handle something that complex and time-varying.
  - His network is **not just sequential**, one layer after another, because it has **two layers working in parallel**.
  - It is called **recurrent** because it **takes its output and sends it back into part of its input**. It produces something at one moment, then uses that to predict the next moment, and then the moment after that.
- **GPT works the same way, one word at a time.** It takes the words of the prompt and predicts the next word, which becomes the first word of its answer. Then it feeds the prompt plus that word back in and predicts the next word again, and keeps going. So the idea behind GPT starts in 1985 with thinking about how speech unfolds in time.

    - People kept refining the recurrent network from 1985 until **2017**, when the **large language model** was invented.
- **Science and technology evolve from other people's ideas over long stretches of time.** An idea from 1943 was refined into the networks of 1986, and Jordan's idea from 1985 was refined into the large language models of 2017. The people who came up with these ideas were mostly not the ones who made a lot of money from them.
    - These years give a sense of the history, but they will not be asked about.

Section 3: From One Neuron to Agentic Systems

- AI systems build up in steps, and **each step is made of the one before it**.
    - **A single neuron takes in a vector and gives back one number.** It multiplies each input by its weight and adds them all up, which is the **inner product**, and the symbol **sigma** just means add them up. This uses nothing but second-grade multiplication and addition to model the electrochemical activity of a real neuron.
    - **A neural network takes in one vector and gives back a whole decision vector.** Its layers are linked by sets of weights, and each set turns one vector into the next through many inner products.
- **A large language model takes in many vectors, one for each moment in a sequence, and gives back one decision vector about what comes next.** Picture 15 vectors, each standing for something at one moment, such as one word of a sentence. The job of the model is to **predict the sixteenth**.
    - In real systems, the vectors number in the hundreds of thousands to a million, and that set is called the **context window**. Each vector is also thousands of numbers long, which is why huge data centers are needed.
    - The sixteenth vector is a **probability distribution over the next word**. With about 30,000 English words, each entry gives the likelihood that one particular word comes next.
- **A large language model is made of neural networks.** Each set of weights is a **layer**, and real systems have about a hundred or more layers. **The knowledge is still in the weights**, so an LLM's knowledge of how to write an essay is stored in its weights.
- **An LLM is a dynamical system**, and that is why the Nobel Prize for AI was awarded in physics. A mass-spring system starts from **initial conditions** and the system decides what happens next. An LLM's initial condition is its set of input vectors, and each layer turns them into **better vectors**, step after step, until the last layer can predict the next word.
    - The calculation at every step is the **inner product**, multiplying and adding billions or trillions of times.
    - In plain terms, an LLM is like the oscillator, except that instead of two numbers changing over time, thousands of numbers are refined layer by layer.
- **An LLM learns by predicting the next word and correcting its mistakes.** Given "the cat sat on the," with **mat** as the real next word, a new LLM with random weights might predict something impossible like "in." Backpropagation adjusts its trillions of weights so the prediction moves toward mat.
    - Repeating this with about a trillion sentences, the system surprisingly ends up knowing how to speak English. No philosopher or cognitive scientist would have expected that human language could be emulated with inner products and functions like the sigmoid.

    - The first problem this design solved, in 2017, was **machine translation**, and the same design then worked for speech and many other problems.
- **An agentic system is a graph of LLMs**, and an **agent** in AI today is just an LLM.
    - An **outline LLM** receives the prompt and is instructed only to outline the answer.
    - A **writing LLM** turns the outline into smooth sentences.
    - A **critic LLM** criticizes the draft, and a **revision LLM** decides whether it is good enough or sends it back to the critic. The back-and-forth is usually limited to a few rounds.
    - A **final LLM** produces the output the user sees.
    - The critic is what greatly reduced **hallucination**, the system making things up, and the inner conversation between LLMs is why many systems take a while to respond.
- The layers of this build-up mirror living things. **Neurons make neural networks, neural networks make LLMs, and LLMs make agentic systems**, much as molecules make cells and cells make organs.
    - Designing a graph of LLMs to solve a problem is a real job today, and it needs problem solving more than knowledge of programming languages.
    - The connections between LLMs can also form circuits nobody planned, which let a system call on other LLMs and do things that were never part of the design, sometimes harmful ones. Limiting those connections is a problem that needs creative people from many fields.
- **The chief insight behind all of this is Gestalt completion.** People see a triangle in the Gestalt figure even though none is drawn, because a big part of human knowledge is **completing** what is missing. Training a system only to **predict the missing thing** turns out to emulate human cognition.
    - The same idea is behind the **cloze procedure**, a test introduced by **Wilson Taylor** in **1953**, in which words are deleted from a text and people fill them in. The name comes from **closure**, the Gestalt term for completing a familiar pattern that is not quite finished, such as seeing a broken circle as whole.
    - **Knowledge can be tested by asking someone to complete a sequence, and for AI, completion is all that knowledge is.** An LLM predicting the next word is playing a cloze game over and over.
    - Underneath everything, **inference is inner products**, and **learning is stochastic gradient descent plus dynamic programming**.

Section 4: Fifteen Vectors of Anything

- **An LLM takes 15 vectors and predicts the sixteenth**, and since a vector is just a list of numbers, those numbers can stand for almost anything. Whatever they stand for, the machine plays the same game of predicting the next one.
- The vectors can be **words**. In "The cat that the dog chased ran up the old tree and sat on the," the sixteenth vector is the missing next word, and a speaker of English knows a likely completion, such as branch or mat.
    - Getting it right means using the whole sentence, since it was the cat, not the dog, that sat.
- The vectors can be **slices of sound**. A spectrogram is cut into little frames, each frame becomes one vector of frequencies and amplitudes, and the system learns to predict the next little piece of speech from all the previous pieces.
    - This approach, in 2019, gave the **best speech recognition in 75 years**, and it is also the basis for synthetic voices that are now very hard to tell from humans.

    - The newest systems can hold a spoken conversation, and the only giveaway left is a slight delay before each response. As that delay shrinks, phone scams become a real danger.
- The vectors can be **pieces of an image**. An image is cut into **16 patches** read row by row, and the system predicts the missing patch. Trained on huge numbers of images, it learns what a dog, a dolphin or a boat is, and this became the basis of modern **computer vision**.
- The vectors can even be **amino acids**. A protein is a string of amino acids, which come in **20 kinds**, and the order of the string decides how the protein folds into its shape. Treating a protein as just a sequence and playing the same predict-the-next game pushed the **protein folding problem** many years ahead, which matters for designing drugs and vaccines.
    - That work won the **2024 Nobel Prize in Chemistry**.
- **The LLM only ever sees vectors.** Words, sounds, pixels and molecules all reach it the same way, so one design asks one question, what comes sixteenth, every time.
    - Because these systems predict from past data, something new that the past data does not cover can make them **hallucinate**. A forecast far into an unfamiliar future can end up as a guess, or as an honest "I don't know."

Section 5: Which Sequences Can Be English Words?

- English allows only **certain sequences of sounds** in its words, and knowing which sequences are allowed is part of every speaker's knowledge of the language. **A simple English syllable can have a consonant, then a vowel, then a consonant**, as in Don, mat, sit, cup, dog and fish.
    - The pattern is about **sounds, not letters**. The silent e in shave, kite and rope does not count, and two letters can spell one consonant sound, as in ship, chin and bath.
- Syllables can also have **clusters**, which are groups of consonants together. A cluster before the vowel is an **onset**, as in split, flat and black, and a cluster after the vowel is a **coda**, as in desk, help and lamp. **Only certain sequences are allowed before the vowel and after the vowel.**
    - Each real word has an impossible twin marked with a star, such as **flat** and **lfat**, **help** and **hepl**, and **lamp** and **lapm**. Many twins use the same letters in the other order, so order matters.
    - **The order mirrors around the vowel.** In flat, the l sits right next to the vowel with the f farther out. In help, the l again sits next to the vowel, now on the other side, with the p farther out. Each impossible twin puts the wrong consonant against the vowel.
    - A made-up product name like "desb" or "lapm" sounds instantly un-English, because it breaks these rules.
- **A long word is a chain of syllables**, such as ba-na-na, At-lan-tic and ath-lete. The sounds t and l are fine across a syllable break, but **tl cannot start a syllable**, which is why Atlantic is a word but tlantic is impossible.
    - The sound **ng** can end a syllable, as in ring, but it can never start one.
    - So whether a sequence of sounds is allowed depends on where the syllable breaks fall.
- **Every language has its own rules.** Mandarin allows only nasal sounds at the end of a syllable, Cantonese also allows a final p, as in cup, and Russian and Polish allow clusters like zb or dl at the start of a word, which English never does. Spanish speakers turn a word like stadium into one that starts with e, so that each syllable avoids an s-t onset.

Section 6: Guessing the Next Letter

- **Phonotactic knowledge can be revealed by the same next-item game that GPT plays.** Each item is the start of an English word, and the task is to guess the next letter. People agree on their guesses quickly, and how much they agree changes from item to item.
- **After a vowel, almost anything can follow.** After "ra," likely letters include t, n, r and l, and many others are possible too.
- **At the start of a word, the choices narrow sharply** to exactly what an English onset allows.
  - After **sp**, the next letter is l, r or a vowel, and never n.
  - After **k**, it is a vowel or n, as in knee, and never t.
  - After **b** and **f**, it is r, l or a vowel, and never n.
  - After **sh**, it is a vowel or r, as in shrimp.
  - After **skr** or **m** at the start of a word, only a vowel can come.
  - After **bram**, the letter b is almost forced, as in bramble, though a new syllable could also start there.
- **Guessing the next letter is predicting the sixteenth item**, the same next-token game an LLM plays. A GPT trained only on this game learns English sounds and spelling, because its predictions can only get good by matching the patterns of English syllables.
  - Its prediction is a **probability distribution** over the next letter, and the shape of that distribution carries the knowledge.
  - After "ra," the distribution is **flat**, spread over many letters, while after "skr" it is **sharply peaked** on a few vowels, because English allows nothing else there.
  - In plain terms, the more the letters so far restrict what can come next, the more the probability piles onto a few letters.

Section 7: Building an LLM Together

- English syllables are a good test case for understanding LLMs. English has about 30,000 words, so training on words needs enormous data, but it has only about 26 letters, or a few dozen sounds. So a small LLM, with thousands of weights instead of trillions, can be trained to learn the knowledge of English syllables that every speaker already has.
- Instead of writing that program by hand, which would take months, the plan is to **design an agentic system using instructions written in English**. The agentic system builds the LLM, trains it to predict each next letter, and shows what it has learned along the way, so it can be compared with the knowledge of English syllables that speakers carry in their heads.

Course: LING285: Human Language and Technology
Title of Lecture: Lecture 18
Date: Thursday, November 5
Professor: Khalil Iskarous

Section 1: From a Neuron to Agents That Communicate

- Everything in modern AI grows from **one very simple idea**. A **neuron** takes information from other neurons, multiplies each input by its weight and adds them up, which is the **inner product**. That arithmetic stands in for neurons communicating electrochemically. People who move science forward usually start from extremely simple ideas like this one and then multiply them many times over.
  - A **neural network** repeats that operation layer after layer, taking a vector in and giving a **decision** out. Even something as complex as a person making a decision can be built from inner products and a function like the sigmoid, combined many times.
  - But a network that makes one **static decision**, such as hearing one vowel from two formants, cannot capture cognition that **unfolds in time**.
- **Language depends on long-distance relationships between words.** In "the dog that my parents, who I love very much, who live in a state far, far away," the next word is **barks**, not bark.
  - **Barks** goes with the singular **dog**, no matter how many other words come in between, even plural ones like parents.
  - No one teaches this rule explicitly. Speakers absorb it from the language around them, and an LLM learns it the same way, from data, by using its weights to do inner products, then more inner products, until it predicts barks.
  - English sentence structure, or **syntax**, is so complicated that many linguists spend their careers on it, and that is why an LLM needs so many layers of computation.
- **LLMs can be multilingual and multimodal.** The first use of LLMs, in 2017, was **translation** between languages, which is now nearly solved for hundreds of languages.
  - A system trained on several languages, such as English, Dutch and Japanese, learns how words go together in general, so it also gets good at very different languages, such as Navajo.
  - **Multimodal** systems connect different modes, such as text, speech and images, so they can caption a photo or create an image from a spoken description.
- **Nobody fully understands what all the weights are doing.** The designs were not planned from scratch. Over about 75 years, people tinkered with something that worked a little, changed it so it worked a little better, and kept going, much as dogs were bred over a few hundred years into today's breeds. Researchers around the world are now trying to figure out what the weights do, partly so that much smaller, less wasteful systems could be built.
- **An agentic system is a team of LLMs that talk to each other in English.** An outline LLM plans the answer, a writing LLM writes it, a critic LLM checks whether each statement follows from the one before, and a revision LLM loops with the critic, sometimes under a **controller** agent that passes work between them.
  - The agents are also **multilingual in computer languages**, such as Python and database languages, because they were trained to predict whole programs too.

- Once each LLM is trained, its weights are usually **not trained further** while the agents work together. Some new systems are starting to learn their own connections, which makes them partly self-designing.
- Agents share a **common memory** where they can write and read information. That shared memory is a big source of their power and also of the danger, because it lets them coordinate in ways nobody planned.
- The same tools can do good work. One agentic system acts as a patient teacher for a nearly lost Native American language, even though there is far too little text in that language to train an ordinary LLM on it.

Section 2: Parallel Heads and Successive Layers

- **Predicting the next word is so difficult that an LLM needs many layers, and in each layer it needs many parallel heads.** A head is a separate expert, and in each layer the knowledge from all the heads is brought together to produce **improved vectors**. Then the same thing happens again in the next layer.
  - This is a question for the exam, so the structure matters. **Layers go one after another from the input to the output, and heads sit side by side within each layer.**
- **A classroom comparison makes heads and layers clear.** Imagine trying to improve oneself by visiting a series of classes, one after another.
  - In the first class, each student is asked for an independent opinion on how to improve. Each student is like a **head**, an expert with its own view.
  - All of those opinions are then **combined** to make one improved version. That combining is what a layer does with the opinions of its heads.
  - Then the next class is visited, and the same thing happens again. **The classes are the layers.**
  - In plain terms, layers give an LLM its depth, one round of improvement after another, and heads give it width, many experts consulted at once within each round.
- **Each head is a set of weights**, and an expert's knowledge is in its weights. Every head in every layer learns how to improve the vectors so that, at the very end, the next word can be predicted.
  - Large systems have **a hundred or more layers** and on the order of **a hundred heads** in each layer, which shows how complicated language, vision and the other things these systems handle really are.
  - The heads and layers are trained the same way as everything else. A sentence like "the dog that ... barks" is taken from text, the system predicts the next word, and if it predicts the wrong word, **every head in every layer** adjusts its weights through backpropagation so the next prediction is closer to barks.
  - Doing this at scale uses enormous resources. In some states, a large share of the water goes to the data centers that run this training.

Section 3: A Dynamical Rule Across Time and Depth

- **A dynamical system is a rule that is fed back to itself.** Numbers go in, the rule acts on them, numbers come out, and the output is fed back in as the next input, again and again. The overall goal is to understand the general structure, not to memorize formulas.
  - In the **harmonic oscillator**, the two numbers are the position x and the velocity v. Starting with x at one and v at zero, the first step leaves x the same and makes v go down.

    - x and v **take turns**. The new x is computed from the old v, and then the new v is computed from that new x. Step by step, the rule draws a **cosine** for x.
    - Fields as far apart as finance use the same idea with different rules. Stock prices are modeled with equations that are fed back on themselves, though the real market never follows them exactly.
- **Jordan's network was a marriage of neural networks and dynamical systems.** Based on the time variation in speech production, Jordan designed a network whose **output is copied back into its state units**, which also keep a fading copy of their own previous values.
    - **Plan units** and **state units** feed the hidden layer, the hidden layer feeds the output, and the output loops back into the state. The same weights are used at every step.
    - After more than **thirty years of tinkering**, this idea became the **LLM**.
- **In an LLM, the layer is the rule.** Each layer applies its weights to every vector, and the next layer works on the result, so **depth plays the role of time**. Following one number up through the layers traces out a curve, much like the oscillator's curve, though not an exact cosine.
    - So the LLM is an unrolled recurrent network, laid out layer by layer, just as the spring's motion was laid out step by step when it was computed on a computer.
    - The rule in each layer is the weights of its heads, and those weights are learned through backpropagation by predicting the next word.
    - Agentic systems also contain loops, such as the critic and revision loop, so dynamical systems run all the way up through AI.
    - The mathematics of inner products that LLMs depend on was first studied deeply in **quantum mechanics**, about a hundred years ago, so physics and AI are closely tied in more ways than one.

Section 4: One Prediction Task for Different Sequences

- An LLM is given **15 vectors** and asked to predict the **16th**, and the vectors can come from anywhere, because the machine only ever sees lists of numbers.
    - They can be **15 words** of a sentence, such as "The cat that the dog chased ran up the old tree and sat on the," with the next word missing.
    - They can be **15 frames of a spectrogram**, with the next slice of speech missing.
    - They can be **15 patches of an image**, read row by row, with the bottom-right patch missing.
    - They can be **15 amino acids** in a protein. In the B chain of insulin, the residue that follows the first fifteen is the amino acid written Y.
- **The same design and the same prediction game work for language, speech, vision and proteins.** Whatever the vectors stand for, the job is always to predict what comes next in the sequence.

Section 5: Which Sequences Can Be English Words?

- English has about 30,000 words, so training an LLM on words takes enormous data, money and energy. But speakers also know a much smaller system made of **only a few dozen sounds**, which is how vowels and consonants can follow each other to make words. Because there are so few sounds, an LLM for this system can be small enough to study closely.

- **A simple English syllable is a consonant, then a vowel, then a consonant**, as in Don, sit, shave, cup, dog and fish. The pattern is about sounds, so the silent e of shave does not count.
  - English also allows **clusters**, as in split, flat, black, glad, desk, help and milk. Only certain clusters are allowed, so **sblit**, **lfat**, **desb** and **lapm** are impossible, and a product called "desb" would sound instantly un-English.
  - Long words are chains of syllables. **Atlantic** is fine because t and l sit across a syllable break, but **tlantic** is impossible, because tl cannot start an English syllable.
- **Every language has its own rules for sequences**, and speakers absorb them without being taught.
  - **Mandarin** allows only nasal sounds at the end of a syllable, so words like mat, sit and shave are impossible in it.
  - **Cantonese** also allows a final p, as in cup.
  - **Russian and Polish** allow clusters like zb or dl at the start of a word, which English never allows.
  - **Spanish** does not allow an s and t to start a word, so speakers add an e at the start of a word like stadium, keeping each syllable simple.

Section 6: Guessing the Next Letter

- Guessing the next letter of an English word reveals this sound knowledge directly. **After a vowel, almost anything can follow**, but **at the start of a word the choices narrow sharply**.
  - After **sp**, only l, r or a vowel can come, and after **skr** or a word-initial **m**, only a vowel can come.
  - After **k**, a vowel or n can come, as in knee, and after **sh**, a vowel or r, as in shrimp.
- **Guessing the next letter is predicting the 16th vector.** A GPT that only plays this game learns English sounds and spelling. Its predicted probability distribution is **flat** after a vowel and **sharply peaked** after skr, because English allows almost nothing else there.

Section 7: Building an LLM with an Agentic System

- An LLM that learns English syllables can be built by **describing it in English** to an agentic system, instead of programming it by hand. The request specified **five layers**, **four heads in each layer**, and **input vectors 64 numbers long**.
  - It asked the system to pick **1000 English words at random** and train the LLM on them, taking each letter and trying to **predict the next letter**, then improving the weights to predict better.
  - It also asked the system to show **ten generated words after each step of training**, to reveal what the LLM currently knew about English.
- The agentic system understood the request in its own technical terms. It built a small **character-level transformer**, another name for an LLM that works letter by letter, with **attention heads**.
  - **Attention** means a head consults the rest of the sequence to work out which preceding items matter, just as predicting barks requires attending to dog.
  - It chose to train on **pronunciations** instead of spellings, so the silent e in shave would not mislead it, and it wrote the program in Python on its own.

- The results show how hard the problem is. After training on 1000 words, the generated words were **still mostly gibberish**, with perhaps a few vowels starting to appear in the right places. Training on **10,000 words** and for many more steps was the suggested next change. A choice that seems easy for any speaker, such as which sounds can follow which, is not easy even for an LLM.
- This way of working has a name.
  - **Software 1.0** is writing programs in a programming language such as Python or C.
  - **Software 2.0** is giving a neural network input data and output data, so the program becomes the **weights** that predict the outputs from the inputs.
  - **Software 3.0** is using an agentic system, a set of LLMs talking to each other and to the user, to build and run a program. Doing this with plain English instructions is called **vibe coding**, and it means someone with a good idea no longer needs to hire a programmer to build it.
- The first attempt at this kind of problem was in **Claude Shannon's 1948 paper, "A Mathematical Theory of Communication."** Working by hand, Shannon generated sequences of letters that came closer and closer to English, from random strings of consonants to strings with vowels and real short words.
  - That paper founded **information theory**, one of the most important fields in engineering, and it remains important for LLMs today.
  - **Viterbi**, one of the developers of dynamic programming, published his most famous work in an information theory journal, so the ideas behind backpropagation and LLMs share the same roots.